Facultad de Ingeniería
Departamento de Ingeniería en Computación

Para la obtención del título
Ingeniero en Computación

'Sistema de gestión de préstamos e inventario del laboratorio del departamento de ingeniería en computación'

Por Sr. Ariel López S.
Profesor guía: Dr. Pedro Alberti V.

10 de abril, 2026.

Índice

1. Introducción **5**

2. Objetivos **6**

3. Motivación y planificación **7**

3.1. Motivo de desarrollo del sistema 7
3.2. Tecnologías a utilizar 8
3.3. Requerimientos 9
3.4. Planificación 10

4. Marco teórico **11**

4.1. PostgreSQL 11
4.2. Python + SQLAlchemy 12
4.3. Alembic 12
4.4. Litestar 13
4.5. Vue3 + TypeScript 14
4.6. PrimeVue + Vite + Pinia + TailwindCSS 14
4.7. Gestión de entorno: UV + Bun 16
4.8. Git + Github 17
4.9. Docker Compose 17

5. Diseño del modelo de base de datos **18**

5.1. Grandes grupos funcionales 18
5.2. División en tablas modulares 20
5.2.1. Módulo de Usuarios 20
5.2.2. Módulo de Autenticación 22
5.2.3. Módulo de Catálogo 22
5.2.4. Módulo de Productos 25
5.2.5. Módulo de Dispositivos 27
5.2.6. Módulo de Prestamos 28
5.3. Modelo de base de datos 30
5.3.1. Diagrama Entidad Relación 31

6. Arquitectura del sistema **32**
6.1. Arquitectura del backend 32
6.2. Arquitectura del frontend 33
6.3. Flujo de consumo de la API 35

7. Implementación de base de datos y backend del sistema **38**
7.1. Uso de Alembic a lo largo del desarrollo 38
7.2. Módulo de Usuarios 38
7.3. Módulo de Autenticación 40
7.4. Módulo de Catálogo 41
7.5. Módulo de Productos 41
7.6. Módulo de Dispositivos 42
7.7. Módulo de Préstamos 44

8. Implementación del frontend del sistema **46**
8.1. Fase 1: Módulo de Usuarios y Autenticación 46
8.2. Fase 2: Módulo de Catálogo 48
8.3. Fase 3: Módulo de Productos 48
8.4. Fase 4: Módulo de Dispositivos 49
8.5. Fase 5: Módulo de Préstamos 50
8.6. Fase 6: UX, UI 51

9. Entorno de ejecución **52**
9.1. Contenedorización con Docker 55

10. Trabajo futuro **60**
10.1. Mejoras al proyecto actual 60
10.2. Nuevos módulos propuestos 62
10.2.1. Préstamo de Inventario (continuación) 62
10.2.2. Administración de Inventario 63
10.2.3. Gestión de DHCP 63
10.2.4. Administración de Salas 64
10.3. Resumen y recomendaciones 65

11. Conclusión **66**

Bibliografía **67**

A. Evidencias de la implementación **72**

Índice de figuras

1. Diagrama Entidad-Relación del modelo de base de datos LabDIC. 31
2. Flujo de consumo de la API del sistema LabDIC. 37
3. Ciclo de vida de una solicitud de préstamo en LabDIC. 50
4. Vista de autenticación — pantalla de inicio de sesión 72
5. Vista de inicio para el rol administrador 73
6. Vista de inicio para el rol usuario 73
7. Vista de administración de usuarios 74
8. Vista de perfil del usuario autenticado 74
9. Vista de catálogo — sección de roles 75
10. Vista de catálogo — sección de ubicaciones 75
11. Vista de catálogo — sección de estados 76
12. Vista de catálogo — sección de modelos 76
13. Vista de administración de productos 77
14. Vista de administración de productos — formulario de nuevo producto 77
15. Vista de administración de productos — detalle de producto . . 78
16. Vista de administración de dispositivos 78
17. Vista de administración de dispositivos — formulario de nuevo dispositivo 79
18. Vista de dispositivos filtrada desde detalle de producto 79
19. Vista de catálogo de dispositivos disponibles para usuarios 80
20. Vista de catálogo de dispositivos — formulario de solicitud de préstamo 80
21. Vista de administración de solicitudes de préstamo 81
22. Vista de administración de préstamos — ciclo de vida en estado inicial 81
23. Vista de administración de préstamos — ciclo de vida finalizado 82
24. Vista de solicitudes del usuario autenticado 82
25. Vista de solicitudes del usuario autenticado — detalle de solicitud 83

Índice de cuadros

1. Módulos funcionales del modelo de datos de LabDIC. 19
2. Estructura de la tabla `users`. 21

3. Estructura de la tabla intermedia `user_roles`. 22
4. Estructura de la tabla `roles`. 23
5. Estructura de la tabla `brands`. 23
6. Estructura de la tabla `models`. 24
7. Estructura de la tabla `categories`. 24
8. Estructura de la tabla `statuses`. 24
9. Estructura de la tabla `ubications`. 25
10. Estructura de la tabla `products`. 26
11. Estructura de la tabla `devices`. 27
12. Estructura de la tabla `device_status_logs`. 28
13. Estructura de la tabla `loan_requests`. 29
14. Estructura de la tabla `loan_request_items`. 30
15. Tecnologías y versiones del proyecto LabDIC. 53
16. Resumen de módulos de trabajo futuro propuestos. 66

1. Introducción

El laboratorio del Departamento de Ingeniería en Computación (LabDIC) de la Universidad de Magallanes gestiona actualmente los préstamos de material tecnológico mediante formularios impresos y registros manuales, lo que dificulta el control, genera duplicidad de información y limita la trazabilidad. Ante esta situación, el presente proyecto propone el desarrollo de una plataforma web que digitalice estos procesos, mantenga actualizada la base de datos del inventario y permita a encargados y solicitantes gestionar las solicitudes de préstamo de forma eficiente. El primer paso consistió en establecer las bases conceptuales, tecnológicas y de planificación que orientan el desarrollo, incluyendo la selección del stack tecnológico, el análisis de requerimientos funcionales y no funcionales, y la estructuración del trabajo en hitos progresivos.

El diseño del modelo de datos partió de tres grandes ejes funcionales —usuarios, productos y préstamos— que, al profundizar en las interacciones entre entidades, derivaron en seis módulos: Usuarios, Autenticación, Catálogo, Productos, Dispositivos y Préstamos. Este diseño modular, implementado con SQLAlchemy como ORM y Alembic para el control de migraciones, garantiza que el esquema pueda evolucionar de forma documentada y reproducible a lo largo de todo el ciclo de desarrollo.

Antes de abordar la implementación de cada módulo, se documentan las decisiones arquitectónicas que estructuran el sistema. En el backend, se define la separación de responsabilidades entre DTOs, repositorios y controladores agrupados bajo el concepto de *servicio*; en el frontend, se documentan las decisiones equivalentes: Vue Router para navegación y control de acceso, Pinia para estado global persistente, y las carpetas `types/` y `services/` como puente tipado hacia la API. El flujo de consumo de la API sintetiza visualmente cómo los datos recorren el sistema completo, desde PostgreSQL hasta los componentes de la interfaz.

La implementación del backend desarrolló los servicios, repositorios, DTOs y controladores para los seis módulos funcionales, exponiendo una API REST completa y documentada. El proceso no fue lineal: a medida que cada módulo tomaba forma, surgieron ajustes en el modelo de datos gestionados mediante migraciones con Alembic, evidenciando la naturaleza iterativa del desarrollo.

El frontend se construyó en seis fases progresivas que van desde la autenticación y control de acceso hasta la gestión completa de préstamos y la revisión de

UX y UI. Durante esta etapa se confirmó la interdependencia real entre ambas capas del sistema: la implementación de vistas de productos y dispositivos reveló la necesidad de correcciones en cascada en la base de datos. Finalmente, se documenta el trabajo futuro, identificando mejoras pendientes al sistema actual y proponiendo nuevos módulos que la arquitectura modular de la plataforma está preparada para recibir.

2. Objetivos

Objetivo general

Desarrollar un sistema web de gestión de préstamos e inventario para el laboratorio del Departamento de Ingeniería en Computación, que digitalice y centralice los procesos de administración del inventario y el ciclo de vida de los préstamos, mediante una arquitectura cliente-servidor modular con control de acceso diferenciado por roles.

Objetivos específicos

- Establecer las bases conceptuales y tecnológicas del proyecto, identificando las herramientas adecuadas para el sistema y analizando las necesidades del laboratorio que el sistema busca resolver.
- Diseñar e implementar un modelo de base de datos relacional que sustente la gestión de usuarios, catálogo de referencia, productos, dispositivos y préstamos, garantizando consistencia referencial, escalabilidad y trazabilidad mediante el control de versiones con Alembic.
- Implementar el backend completo del sistema, desarrollando los módulos funcionales de Usuarios, Autenticación, Catálogo, Productos, Dispositivos y Préstamos mediante servicios, repositorios, DTOs y controladores, exponiendo una API REST documentada que sirva de base para el desarrollo del frontend.
- Desarrollar el frontend completo del sistema a través de fases progresivas, implementando las vistas, componentes, stores y servicios para que administradores y usuarios interactúen con todos los módulos mediante una interfaz funcional, responsiva y con control de acceso basado en roles.

- Documentar la arquitectura técnica del sistema, describiendo la separación de responsabilidades entre las capas del backend, el flujo de consumo de la API y los mecanismos de gestión de estado y control de acceso implementados en el frontend.

3. Motivación y planificación

3.1. Motivo de desarrollo del sistema

El laboratorio del Departamento de Ingeniería en Computación cuenta con personal encargado de diversas tareas administrativas, entre las que destacan la organización de salas, la revisión de equipos, la atención de requerimientos técnicos y la gestión de objetos extraviados. Sin embargo, una de las labores más demandantes corresponde al préstamo de material, que incluye notebooks, periféricos, unidades USB, libros de estudio, adaptadores y cables de red.

Actualmente, este proceso se realiza de forma manual mediante formularios impresos, en los cuales los solicitantes deben registrar datos personales y de contacto. Dichos documentos, firmados por el encargado del laboratorio, sirven como respaldo tanto para la apertura como para el cierre de los préstamos. Este método genera un alto volumen de papeleo, dificulta la trazabilidad y depende exclusivamente de la disponibilidad física de los formularios.

Por otra parte, el laboratorio dispone de un archivo Excel que contiene el inventario de los equipos y recursos, pero este permanece inactivo, sin integrarse al proceso de control ni a la gestión de préstamos. La falta de actualización y uso de este recurso evidencia una oportunidad de mejora significativa en la administración del inventario.

El uso de bases de datos relacionales, una arquitectura modular y escalable, y una aplicación web accesible desde la red interna garantiza la trazabilidad, seguridad y proyección futura del sistema. De esta manera, la digitalización de los procesos permitirá contar con una herramienta confiable y adaptable, que no solo resuelva las necesidades actuales del laboratorio, sino que también siente las bases para incorporar nuevas funcionalidades y responder a requerimientos futuros.

3.2. Tecnologías a utilizar

Para el desarrollo del sistema se definió un conjunto de tecnologías que permiten garantizar robustez, escalabilidad y facilidad de mantenimiento, abarcando tanto la gestión de datos como el desarrollo de la aplicación web. A continuación, se describen los principales componentes del stack tecnológico seleccionado, y su profundización estará presente en la sección 4, a través del marco teórico:

Backend

- **Base de datos**: PostgreSQL.
- **Gestión de versiones de la base de datos**: Alembic.
- **Lenguaje de desarrollo**: Python
- **ORM**: SQLAlchemy
- **Framework**: Litestar

Frontend

- **Framework**: Vue 3
- **Lenguaje de desarrollo**: TypeScript
- **Diseño y componentes UI**: PrimeVue + TailwindCSS
- **Gestión de estado**: Pinia
- **Herramienta de construcción y servidor de desarrollo**: Vite

Gestión

- **Gestión de entorno backend**: UV.
- **Gestión de entorno frontend**: bun.
- **Gestión de versiones del proyecto**: Git + Repositorio en GitHub.

El sistema se estructura bajo un modelo de arquitectura cliente-servidor, donde el frontend actúa como cliente que interactúa con los usuarios, el backend expone servicios mediante una API RESTful, y la base de datos se encarga de la persistencia de información. Este enfoque modular asegura que cada capa pueda evolucionar de manera independiente y facilita la integración futura de nuevas funcionalidades.

3.3. Requerimientos

El sistema responde a la necesidad de digitalizar, agilizar y reducir los errores humanos en la gestión de préstamos e inventario del laboratorio de computación. Para ello, se definen los siguientes requerimientos:

Requerimientos Funcionales (RF)

- **RF1. Registro de préstamos**: El sistema debe permitir registrar préstamos en tiempo real, incluyendo usuario solicitante, material prestado y fechas de entrega y devolución.
- **RF2. Gestión de inventario**: El sistema debe mantener un inventario dinámico, que se actualice automáticamente con cada préstamo o devolución, reflejando siempre el stock real.
- **RF3. Gestión de usuarios**: El sistema debe permitir la creación, autenticación y administración de usuarios con contraseñas cifradas.
- **RF4. Control de acceso**: El sistema debe implementar permisos diferenciados según el rol del usuario (Estudiante, Profesor, Administrador, Invitado).

Requerimientos No Funcionales (RNF)

- **RNF1. Seguridad**: El sistema debe utilizar técnicas de cifrado seguro para contraseñas (hashing con `pwdlib`) y garantizar la integridad de la información.
- **RNF2. Escalabilidad**: El sistema debe estar diseñado de manera modular para permitir la incorporación de nuevos módulos en el futuro sin afectar la estabilidad.
- **RNF3. Mantenibilidad**: El sistema debe contar con documentación técnica actualizada que facilite futuras mejoras y correcciones.
- **RNF4. Rendimiento**: El sistema debe garantizar tiempos de respuesta nativos menores a 1.5 segundos, permitiendo la gestión simultánea de múltiples solicitudes sin afectar la experiencia del usuario.

- **RNF5. Interfaz de usuario**: El sistema debe contar con una interfaz gráfica intuitiva, que facilite la interacción tanto a usuarios como a administradores.

- **RNF6. Usabilidad**: La interfaz debe ser accesible, consistente y simple de utilizar, reduciendo la curva de aprendizaje de los usuarios.

3.4. Planificación

La planificación del sistema se estructura en cinco hitos, cada uno de los cuales representa una etapa de desarrollo con entregables concretos y funcionales. Esta organización permite dar seguimiento progresivo al avance del proyecto, asegurando que cada fase consolide los fundamentos necesarios para la siguiente.

- **Hito 1 (2 semanas):** Planificación, motivación y marco teórico. Comprende el análisis de las necesidades del laboratorio, la definición del alcance del sistema, la selección del stack tecnológico y la elaboración del marco teórico que fundamenta las decisiones de diseño.

- **Hito 2 (2 semanas):** Diseño e implementación del modelo de base de datos relacional. Incluye la definición de las entidades del sistema (usuarios, catálogo, productos, dispositivos y préstamos), sus relaciones y la configuración del entorno de migraciones con Alembic.

- **Hito 3 (3 semanas):** Implementación del backend. Desarrollo de los servicios, repositorios, DTOs y controladores de los seis módulos funcionales del sistema, exponiendo la API REST documentada que sirve como base para el desarrollo del frontend. A lo largo de este hito se aplicarán migraciones iterativas con Alembic conforme evolucionen los requerimientos.

- **Hito 4 (9 semanas):** Implementación del frontend. Desarrollo progresivo de las vistas, componentes, stores y servicios del sistema, cubriendo los módulos de usuarios y autenticación, catálogo de administración, productos, dispositivos y préstamos. Incluye la revisión final de UX/UI, el soporte de modo claro y oscuro, el comportamiento *responsive* y la implementación de notificaciones `Toast`. Este hito contempla una pausa programada sin avances entre las fases intermedias.

- **Hito 5 (2 semanas):** Documentación de la arquitectura técnica del sistema. Descripción de los patrones de diseño, la organización de capas, los

mecanismos de comunicación entre frontend y backend, y el flujo completo de consumo de la API.

4. Marco teórico

4.1. PostgreSQL

Para el desarrollo del sistema se opta por utilizar PostgreSQL como motor de base de datos relacional, debido a que ofrece un conjunto de características avanzadas que garantizan integridad, eficiencia y escalabilidad en entornos multiusuario [1].

En primer lugar, PostgreSQL asegura el cumplimiento estricto de las propiedades ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad), lo que garantiza que las transacciones sean confiables incluso bajo condiciones de alta concurrencia o fallos inesperados [2] [3]. Esta solidez resulta fundamental para un sistema de préstamos e inventario, donde la pérdida o corrupción de datos comprometería el control de los recursos.

Adicionalmente, PostgreSQL proporciona un soporte amplio de tipos de datos más allá de los estándares SQL, incluyendo JSONB para almacenar y consultar datos semiestructurados, arrays, enumerados y UUID [3] [4]. Estas capacidades permiten modelar de manera más flexible los distintos elementos del inventario y facilitar futuras ampliaciones del sistema sin recurrir a soluciones externas.

Otro aspecto clave corresponde a la optimización de consultas mediante algoritmos de índices avanzados. PostgreSQL soporta estructuras como B-Tree, Hash, GiST, GIN y BRIN, lo que posibilita mejorar el rendimiento en búsquedas complejas, filtrados y operaciones sobre grandes volúmenes de datos [4]. Esta versatilidad en el manejo de índices resulta especialmente ventajosa para mantener la eficiencia del sistema a medida que la cantidad de registros crezca.

Finalmente, la implementación de Multiversion Concurrency Control (MVCC) permite que múltiples transacciones de lectura y escritura se ejecuten de manera simultánea sin generar bloqueos innecesarios [1] [3]. Este enfoque asegura que el sistema pueda atender a varios usuarios en paralelo, manteniendo la coherencia de los datos y mejorando la experiencia de uso.

4.2. Python + SQLAlchemy

La elección de Python y SQLAlchemy no constituye una novedad en el ámbito del desarrollo de aplicaciones, ya que existe una amplia cantidad de documentación, tutoriales y guías en línea que respaldan su uso [5] [6] [7]. Python destaca por su facilidad de aprendizaje y la abundancia de recursos didácticos, lo que facilita la adopción por parte de programadores junior en implementaciones básicas. Al mismo tiempo, su versatilidad y compatibilidad con programación asíncrona avanzada, combinada con frameworks modernos, lo convierten en una herramienta altamente productiva también para desarrolladores senior que requieren mayor control a bajo nivel y optimización en aplicaciones de alta demanda. Esta dualidad entre accesibilidad y potencia explica en gran medida su popularidad y la solidez de su ecosistema. [8] [9]

En cuanto a la capa de acceso a datos, SQLAlchemy se adopta por su equilibrio entre expresividad y control. Aporta dos niveles de abstracción: En primer lugar, *SQLAlchemy Core*, un generador de SQL declarativo y composable que permite optimización fina cuando es necesario, y el segundo consta del ORM (Object Relational Mapper), que modela el dominio en clases y relaciones, favoreciendo consistencia del modelo, reutilización y pruebas. Posee una amplia compatibilidad de dialectos entre ellos PostgreSQL, MySQL/MariaDB y SQLite [10]. Reduciendo el acoplamiento a un motor específico y facilitando portabilidad. Finalmente, la integración nativa con Alembic habilita un flujo de trabajo coherente para la evolución del esquema, con migraciones versionadas y reproducibles.

4.3. Alembic

En el proyecto se utiliza Python como lenguaje de programación para el backend y SQLAlchemy como ORM para la gestión de la persistencia de datos. Bajo este contexto, se ha decidido implementar Alembic como herramienta de migración de base de datos, dado que ofrece un conjunto de características que se ajustan de manera óptima a las necesidades del sistema.

En primer lugar, Alembic permite manejar de forma estructurada la evolución del esquema de la base de datos, lo que resulta esencial en proyectos donde el modelo de datos se encuentra en constante ajuste durante las fases de desarrollo [11]. Esta capacidad evita inconsistencias entre los entornos de desarrollo y prueba, y sienta las condiciones necesarias para una futura transición a un

entorno de producción sin contratiempos.

Alembic facilita la generación y aplicación de migraciones de forma controlada, permitiendo registrar cada modificación en scripts versionados que conforman un historial claro de los cambios realizados en la base de datos [11] [12]. Esta trazabilidad garantiza la consistencia del modelo de datos a lo largo de todo el ciclo de vida del proyecto, agiliza el despliegue de actualizaciones y reduce el riesgo de errores humanos asociados a modificaciones manuales del esquema. Resulta especialmente valioso en entornos colaborativos, donde múltiples desarrolladores interactúan con la misma base de datos y es imprescindible mantener un control preciso de la evolución del esquema.

Finalmente, la compatibilidad nativa de Alembic con Python y SQLAlchemy refuerza su pertinencia en el proyecto [11]. Al integrarse de manera fluida con el stack tecnológico seleccionado, se optimiza la curva de aprendizaje y se garantiza la coherencia entre las distintas capas del sistema.

En conjunto, estas características hacen que Alembic se presente como la opción más adecuada para la gestión de versiones de la base de datos del proyecto, asegurando control, consistencia y escalabilidad en la administración del modelo de datos.

4.4. Litestar

Litestar se distingue por ofrecer un rendimiento superior, alcanzando aproximadamente el doble de velocidad que FastAPI. Esta ventaja proviene en gran medida de su uso de msgspec como motor de serialización, en lugar de Pydantic, lo que reduce significativamente la sobrecarga en el ciclo de solicitud y respuesta. [13] [14]

Para proyectos que utilizan SQLAlchemy como ORM, Litestar incorpora un plugin nativo que simplifica la gestión de sesiones y transacciones, junto con un Repository Patter integrado que facilita las operaciones de base de datos [15]. También incluye una capa de servicio para organizar la lógica de negocio y la transformación de datos, cuenta con paginación y filtrado integrados, y ofrece herramientas CLI que facilitan las migraciones de base de datos. [13] [15]

Otro aspecto relevante es su sistema de inyección de dependencias, inspirado en pytest, que supera en flexibilidad al de FastAPI. Este permite declarar dependencias en distintos niveles de aplicación, router, controlador y handler, lo que brinda un control más granular sobre la arquitectura del proyecto. [16] [14]

Finalmente, Litestar integra de forma nativa diversas funcionalidades que

en otros frameworks requieren librerías externas. Entre ellas se incluyen autenticación JWT y sistemas personalizados de autenticación, soporte completo de CORS, manejo de sesiones con integración a SQLAlchemy, guards de autorización para control de acceso granular y un middleware robusto y extensible. [13] [14] [17]

4.5. Vue3 + TypeScript

Vue 3 es un framework frontend altamente flexible, que fue diseñado para el desarrollo de aplicaciones web modernas y escalables, facilita un código modular, rendimiento optimizado mediante mejoras en compilación y ejecución, y soporte nativo para TypeScript, además, trabaja muy bien con PrimeVue debido a la amplia colección de componentes preconstruidos que ofrece, los cuales aceleran la implementación de funcionalidades y aseguran una interfaz de usuario consistente y responsiva, en conjunto, permite generar código más robusto y detectar errores en etapas tempranas del desarrollo [18].

TypeScript es uno de los lenguaje recomendados y esenciales para el desarrollo frontend, pues, permite detectar errores en tiempo de compilación, reduciendo fallas que de otra forma solo aparecerían en ejecución, además, fomenta una mejor organización del código y soporta la programación orientada a objetos mediante clases e interfaces [19]. Esto aporta claridad, documentación implícita y una mantenimiento mejorado, aunque su uso implica una curva de aprendizaje inicial, los beneficios en calidad y escalabilidad superan ampliamente dichas limitaciones [20].

4.6. PrimeVue + Vite + Pinia + TailwindCSS

PrimeVue se ha consolidado como una de las bibliotecas de componentes más robustas y completas para aplicaciones desarrolladas con Vue.js. Con más de 80 componentes disponibles, abarca desde elementos básicos hasta recursos avanzados como `DataTables`, `TreeTables` y gráficos interactivos, lo que la convierte en una solución integral para proyectos de diversa complejidad. [21]

Un aspecto relevante es que PrimeVue adopta un esquema de versionado semántico, lo que garantiza estabilidad y continuidad a largo plazo, evitando cambios disruptivos que puedan comprometer sistemas en producción. [22]

Otro de sus puntos fuertes es la flexibilidad en el diseño, ya que ofrece soporte tanto para el modo `styled` como `unstyled`, permitiendo integrarse de manera óptima con frameworks de estilos modernos. En particular, la combinación con

TailwindCSS ofrece una experiencia de desarrollo altamente personalizable y consistente, aprovechando la potencia de los estilos utilitarios. [23]

TailwindCSS se ha consolidado como un framework de utilidades altamente eficiente para el desarrollo de interfaces modernas. Su principal fortaleza radica en el compilador Just-in-Time (JIT), que genera únicamente los estilos utilizados en la aplicación y elimina automáticamente el CSS no empleado. Este enfoque permite resultados de alto rendimiento en producción, como lo demuestra el caso de Netflix Top 10, cuya aplicación completa carga apenas 6.5kB de CSS. [24]

Además de su eficiencia, TailwindCSS aporta un conjunto de características que favorecen la mantenibilidad y escalabilidad de los proyectos. Entre ellas destacan el sistema de Design Tokens, que garantiza consistencia en colores, tipografías y espaciados; la abstracción de componentes, que permite construir interfaces modulares a partir de estilos utilitarios; y la optimización de la etapa de compilación (build), que asegura entregas más ligeras y rápidas. Estas cualidades lo convierten en una herramienta estratégica para proyectos que requieren interfaces responsivas, personalizables y con un alto nivel de rendimiento. [25]

Vite se ha consolidado como una herramienta clave en el ecosistema frontend, destacando por la rapidez que ofrece tanto en la etapa de desarrollo como en la construcción de aplicaciones para producción. Distintos análisis la señalan como un verdadero cambio de paradigma, ya que proporciona arranques casi instantáneos, una retroalimentación rápida durante la programación y un flujo de trabajo que mejora notablemente la experiencia del desarrollador. En producción, genera builds altamente optimizadas y ligeras, mientras que en desarrollo permite mantener un ciclo ágil que se adapta a proyectos de cualquier escala, favoreciendo tanto la eficiencia como el rendimiento general del sistema. [26] [27]

Pinia ha reemplazado oficialmente a Vuex como la solución de gestión de estado en Vue 3, consolidándose como una herramienta más moderna y eficiente. Sus mejoras se reflejan en un rendimiento superior, con un 37 % más de rapidez en actualizaciones de estado y un 23 % menos de uso de memoria en aplicaciones complejas. Además, ofrece modularidad nativa sin configuración adicional, lo que simplifica la estructura de proyectos escalables. [22] [28]

En aplicaciones de mayor escala, Pinia aporta características clave como la carga diferida (lazy loading) de stores, que reduce el tamaño del bundle inicial; la persistencia del estado mediante localStorage o sessionStorage; y la facilidad para organizar stores modulares que se cargan bajo demanda. Estas cualidades permiten construir aplicaciones más eficientes, escalables y fáciles de mantener, garantizando un control de estado centralizado sin sacrificar el rendimiento. [29]

La incorporación conjunta de todas estas tecnologías, aporta beneficios significativos para los equipos de desarrollo, potenciando la productividad y la mantenibilidad de proyectos, permitiendo un tiempo reducido de configuración inicial, ofreciendo una experiencia de desarrollo optimizada y promoviendo la reutilización de código, logrando un código base más limpio, organizado y escalable. Además, favorecen una mejor separación de responsabilidades entre capas de la aplicación y simplifican las tareas de pruebas y mantenimiento, consolidando un entorno moderno y eficiente para el desarrollo frontend.

4.7. Gestión de entorno: UV + Bun

UV es un gestor de paquetes y proyectos altamente eficiente y veloz, diseñado por Astral para unificar en una sola herramienta las funcionalidades que tradicionalmente dependen de un ecosistema fragmentado en Python [30]. Así, reemplaza utilidades como `pip`, `pip-tools`, `virtualenv/venv` y `pyenv`, ofreciendo una solución integral para todo el ciclo de vida del desarrollo en este lenguaje. [31]

Su arquitectura introduce innovaciones significativas que optimizan el rendimiento como por ejemplo, permitiendo descargas paralelas de múltiples paquetes, utilizando caché global optimizado con técnicas como `Copy-on-Write` y `hard links` para evitar redundancias, y realiza un manejo inteligente de metadatos, descargando solo la información necesaria, junto a todo esto, sumamos su implementación nativa en Rust, que asegura operaciones compiladas considerablemente más rápidas. [32] [33] Continuando con sus beneficios, en términos de compatibilidad, UV respeta plenamente los estándares oficiales de packaging en Python, empleando `pyproject.toml` como su fuente para administrar paquetes, junto con archivos de bloqueo universales que garantizan reproducibilidad en múltiples plataformas. [34]

Bun es un gestor de paquetes y entorno de ejecución para JavaScript que destaca por su alto rendimiento al utilizar el motor JavaScriptCore de WebKit, en contraste con el motor V8 de Chrome empleado por otros entornos [35]. Esta elección tecnológica le otorga ventajas significativas en tiempos de inicio y en operaciones intensivas de CPU. Diversos benchmarks independientes han demostrado que Bun supera consistentemente a Node.js en múltiples métricas: en tareas de cálculo, como la generación y ordenamiento de 100,000 números aleatorios [36], Bun reduce los tiempos de ejecución a aproximadamente 1,700 ms frente a los 3,400 ms de Node.js [36]; en el rendimiento de servidores HTTP al-

canza alrededor de 52,000 solicitudes por segundo frente a 13,000 de Node.js [35]; y en la instalación de dependencias su gestor de paquetes nativo logra ser hasta 25 veces más rápido que npm, reduciendo de forma drástica los tiempos de configuración de proyectos. [37] [38]

En términos generales, el motivo de uso tanto de Bun en el frontend como de UV en el backend se sustenta en la velocidad, innovación y disrupción tecnológica que ambos están aportando en un corto periodo de existencia. Estas herramientas representan una nueva generación de gestores de entornos y paquetes, capaces de optimizar los flujos de desarrollo sin apartarse de los estándares modernos que guían su evolución, lo que las convierte en una elección estratégica para proyectos que buscan eficiencia y escalabilidad.

4.8. Git + Github

Git como sistema de gestor de versiones y Github como plataforma para llevar a cabo dicha tarea en la nube, son dos elementos fundamental que todo informático debe tener, pues en el desarrollo de proyectos de software son herramientas universales para el trabajo bien empleado e incluso colaborativo. Git permite llevar un control preciso de los cambios realizados en el código, facilitando la trazabilidad de cada modificación y la posibilidad de retroceder a versiones anteriores en caso de errores o conflictos. Esto asegura mayor seguridad en el desarrollo y disminuye riesgos de pérdida de información. [39]

Por su parte, GitHub complementa esta funcionalidad al ofrecer un espacio de almacenamiento remoto y respaldo en la nube, además de un entorno para la colaboración entre equipos de trabajo. Ambos aportan buenas prácticas profesionales, constituyéndose como herramientas indispensables para cualquier programador moderno.

4.9. Docker Compose

Docker Compose es una herramienta esencial para la gestión de aplicaciones que requieren múltiples servicios interconectados, pues, a través de un único archivo de configuración en muchas ocasiones simplemente llamado `docker compose.yml`, es posible definir de manera centralizada los servicios, redes, volúmenes y variables de entorno que conforman la infraestructura de la aplicación. De este modo, se elimina la necesidad de administrar contenedores de forma individual, ofreciendo una visión unificada y ordenada del sistema. [40] [41]

La principal ventaja de Docker Compose radica en la simplificación de la orquestación de contenedores, automatizando tareas que antes demandaban intervención manual [42]. Por ejemplo, en entornos donde se emplean arquitecturas de microservicios, facilita la coordinación entre componentes independientes (bases de datos, APIs, servicios web, entre otros), garantizando a la vez el aislamiento necesario para un funcionamiento seguro y eficiente. [43]

Un aspecto clave para la elección de Docker es su capacidad de asegurar la reproducibilidad del entorno. Gracias a su funcionamiento basado en contenedores, es posible ejecutar la misma configuración de manera idéntica en distintos sistemas [44], lo que resulta especialmente relevante de cara a un eventual despliegue del sistema en los servidores del laboratorio. Este aspecto, que escapa a los límites del presente proyecto, queda respaldado por la elección de Docker Compose como orquestador: el sistema está preparado para ser desplegado en producción en el momento en que se decida avanzar en esa dirección. Así, los problemas recurrentes de reinstalación manual, inconsistencias entre entornos y pérdida de tiempo quedan resueltos de antemano, garantizando un entorno consistente, confiable y fácil de replicar [45].

5. Diseño del modelo de base de datos

5.1. Grandes grupos funcionales

En una primera aproximación, el sistema puede describirse en torno a tres grandes ejes funcionales: los **usuarios** interactúan con el sistema para solicitar un **préstamo** de **productos**, y dichas solicitudes son gestionadas por los administradores hasta concretar la entrega y registrar la devolución.

Sin embargo, a medida que el diseño avanza, la complejidad de las interacciones entre entidades evidencia la necesidad de subdividir estos grandes ejes en módulos más pequeños y cohesivos. Este enfoque de *dividir para vencer* permite que cada módulo tenga una responsabilidad bien definida, reduzca el acoplamiento entre tablas y facilite tanto el mantenimiento como la extensión futura del sistema.

Así, el modelo de base de datos final se organiza en seis módulos funcionales, cuyas entidades y relaciones se describen en el cuadro 1.

Módulo	Propósito principal	Entidades representativas
Usuarios	Gestionar la identidad, credenciales y datos personales de quienes interactúan con el sistema.	`users`, `user_roles`
Autenticación	Controlar el acceso mediante credenciales cifradas y permisos por rol, sin requerir tablas adicionales: se implementa a través de campos específicos del modelo de usuario.	Campos `password`, `is_active`, `is_admin` en `users`
Catálogo	Centralizar los datos de referencia reutilizables por otros módulos del sistema, evitando redundancia y facilitando la administración. Incluye `roles`, que inicialmente pertenecía al módulo de usuarios pero fue reubicado aquí por compartir la misma naturaleza de dato de referencia con funcionalidades CRUD simples y acotadas.	`roles`, `brands`, `models`, `categories`, `statuses`, `ubications`
Productos	Representar los tipos de ítems disponibles para préstamo y sus características descriptivas.	`products`
Dispositivos	Administrar las unidades físicas individuales del inventario, su estado actual y el historial de cambios de estado.	`devices`, `device_status_logs`
Préstamos	Modelar el ciclo de vida completo de las solicitudes de préstamo, desde la solicitud inicial hasta la devolución.	`loan_requests`, `loan_request_items`

Cuadro 1: Módulos funcionales del modelo de datos de LabDIC.

Cabe destacar que la tabla `statuses` es transversal al modelo: sus registros son referenciados tanto por `devices` como por `loan_requests` y `device_status_logs`, lo que refleja que el concepto de *estado* opera como un eje compartido entre los módulos de dispositivos y préstamos.

5.2. División en tablas modulares

Cada módulo funcional se implementa mediante una o más tablas, diseñadas para ser cohesivas internamente y débilmente acopladas entre sí. Las relaciones entre módulos se establecen a través de claves foráneas, preservando la integridad referencial del modelo. Cualquier cambio estructural futuro en el esquema se gestiona mediante migraciones controladas con Alembic, garantizando trazabilidad y consistencia a lo largo del ciclo de vida del proyecto.

5.2.1. Módulo de Usuarios

El módulo de usuarios está diseñado para gestionar la autenticación, control de acceso y roles dentro del sistema. La tabla `users` almacena la información básica de cada usuario, incluyendo sus datos personales, credenciales y el estado de su cuenta. Este modelo permite identificar de manera única a los usuarios mediante su RUT, nombre de usuario o correo electrónico, y controlar el acceso a las funcionalidades del sistema a través de campos como `is_active` e `is_admin`.

Tabla usuarios

Campo	Descripción
`id`	Identificador único del usuario. Es la clave primaria de la tabla.
`rut`	RUT del usuario, utilizado como identificador único. Este campo es obligatorio y único para cada usuario.
`name`	Nombre completo del usuario. Se utiliza para mostrar al usuario su nombre en el sistema.
`username`	Nombre de usuario único para acceder al sistema. Este campo también debe ser único.
`email`	Correo electrónico del usuario, utilizado para la autenticación y la comunicación. Este campo es único.
`password`	Contraseña cifrada del usuario. Es el campo crítico para la seguridad del acceso al sistema.
`phone`	Número de teléfono del usuario. Es un campo opcional.
`address`	Dirección física del usuario. También es un campo opcional.
`created_at`	Fecha y hora en que el usuario fue creado en el sistema. Se asigna automáticamente al momento de la creación.
`is_active`	Indica si la cuenta del usuario está activa. Permite habilitar o deshabilitar el acceso al sistema.
`is_admin`	Define si el usuario tiene permisos de administrador. Es un campo booleano que indica si el usuario tiene privilegios adicionales dentro del sistema.

Cuadro 2: Estructura de la tabla `users`.

Tabla user_roles

La tabla `user_roles` actúa como tabla intermedia que resuelve la relación muchos a muchos entre usuarios y roles. Un usuario puede tener múltiples roles asignados y un mismo rol puede estar asociado a múltiples usuarios. Su estructura es deliberadamente minimalista, limitándose a las claves foráneas necesarias para establecer dicha asociación.

Campo	Descripción
`user_id`	Clave foránea que referencia a `users.id`. Forma parte de la clave primaria compuesta.
`role_id`	Clave foránea que referencia a `roles.id`. Forma parte de la clave primaria compuesta.

Cuadro 3: Estructura de la tabla intermedia `user_roles`.

5.2.2. Módulo de Autenticación

El módulo de autenticación no requiere tablas propias en el esquema relacional. Su implementación se sustenta en campos específicos de la tabla `users`: `password` almacena la contraseña cifrada mediante hashing, mientras que `is_active` e `is_admin` controlan el estado de la cuenta y el nivel de acceso del usuario respectivamente. Complementariamente, la relación muchos a muchos con `roles` a través de `user_roles` permite implementar un control de acceso granular basado en roles.

Este módulo ilustra que la modularidad del sistema no se define exclusivamente por la cantidad de tablas que lo componen, sino por la responsabilidad funcional que agrupa: en este caso, garantizar que solo usuarios válidos y autorizados puedan operar el sistema.

5.2.3. Módulo de Catálogo

Tabla de roles

La tabla `roles` se encarga de almacenar los diferentes roles que los usuarios pueden tener dentro del sistema. Cada rol tiene un nombre único y una descripción opcional que explica sus permisos o funciones dentro del sistema. Esta tabla permite gestionar y asignar roles a los usuarios, facilitando el control de acceso y la administración de permisos.

Campo	Descripción
`id`	Identificador único del rol. Es la clave primaria de la tabla.
`name`	Nombre del rol, utilizado para identificar el rol de manera única dentro del sistema (por ejemplo, 'admin', 'usuario'). Este campo es obligatorio y único.
`description`	Descripción del rol. Proporciona detalles sobre las funciones y permisos asociados con el rol. Es un campo opcional.

Cuadro 4: Estructura de la tabla `roles`.

La gestión de roles es fundamental para la administración de permisos en el sistema. A través de esta tabla, se pueden asignar diferentes capacidades a los usuarios según el rol que se les asigne, asegurando un control adecuado sobre el acceso y las funcionalidades disponibles para cada tipo de usuario.

Tabla de marcas

La tabla `brands` almacena las marcas asociadas a los productos. Cada marca tiene un nombre único, lo que permite categorizar los productos según su fabricante. Esta tabla está relacionada con la tabla `products`, de modo que cada producto puede tener una marca específica.

Campo	Descripción
`id`	Identificador único de la marca. Es la clave primaria de la tabla.
`name`	Nombre de la marca, utilizado para identificarla de manera única dentro del sistema (por ejemplo, 'Dell', 'HP', 'Apple'). Este campo es obligatorio y único.

Cuadro 5: Estructura de la tabla `brands`.

Tabla de modelos

La tabla `models` almacena los modelos de los productos. Cada modelo tiene un nombre único y está relacionado con los productos disponibles en el inventario. Al igual que la tabla de marcas, los productos pueden estar asociados a un modelo específico.

Campo	Descripción
`id`	Identificador único del modelo. Es la clave primaria de la tabla.
`name`	Nombre del modelo (por ejemplo, 'XPS 13', 'Spectre x360', 'MacBook Pro'). Este campo es obligatorio y único.

Cuadro 6: Estructura de la tabla `models`.

Tabla de categorías

La tabla `categories` agrupa los productos en categorías específicas, como 'Laptops', 'Tablets' o 'Smartphones'. Esto facilita la organización y búsqueda de productos dentro del inventario.

Campo	Descripción
`id`	Identificador único de la categoría. Es la clave primaria de la tabla.
`name`	Nombre de la categoría (por ejemplo, 'Laptops', 'Tablets', 'Smartphones'). Este campo es obligatorio y único.

Cuadro 7: Estructura de la tabla `categories`.

Tabla de estados

La tabla `statuses` permite gestionar los diferentes estados de los dispositivos y las solicitudes asociadas, como 'available', 'borrowed', 'under maintenance', entre otros. Los estados se utilizan para rastrear el ciclo de vida de cada dispositivo y solicitud dentro del sistema.

Campo	Descripción
`id`	Identificador único del estado. Es la clave primaria de la tabla.
`name`	Nombre del estado (por ejemplo, 'available', 'borrowed', 'under maintenance'). Este campo es obligatorio y único.

Cuadro 8: Estructura de la tabla `statuses`.

Tabla de ubicaciones

La tabla `ubications` gestiona las ubicaciones físicas de los dispositivos dentro del sistema. Cada ubicación tiene un nombre y una descripción opcional, lo que permite identificar fácilmente el lugar donde se encuentra cada dispositivo.

Campo	**Descripción**
`id`	Identificador único de la ubicación. Es la clave primaria de la tabla.
`name`	Nombre de la ubicación (por ejemplo, 'TI-1', 'Sala de servidores', 'Administración'). Este campo es obligatorio y único.
`description`	Descripción detallada de la ubicación. Este campo es opcional y proporciona información adicional sobre la ubicación.

Cuadro 9: Estructura de la tabla `ubications`.

5.2.4. Módulo de Productos

El módulo de productos es responsable de gestionar el inventario de ítems disponibles para préstamo dentro del sistema. La tabla `products` almacena la información de cada producto, como su nombre, descripción, cantidad disponible y otras propiedades relevantes para gestionar los préstamos de los productos. También se mantiene la relación con otras tablas como marcas, modelos y categorías de productos.

Tabla de productos

La tabla `products` es fundamental para el control del inventario, ya que permite gestionar y actualizar las existencias de los productos disponibles para préstamo. Además, almacena información relacionada con la descripción del producto y su relación con otras entidades, como las marcas y los modelos.

Campo	Descripción
`id`	Identificador único del producto. Es la clave primaria de la tabla.
`name`	Nombre del producto (por ejemplo, 'Laptop Dell XPS 13'). Este campo es único y obligatorio.
`brand_id`	ID de la marca asociada al producto. Es una clave foránea que referencia a la tabla `brands`. Este campo es opcional.
`model_id`	ID del modelo asociado al producto. Es una clave foránea que referencia a la tabla `models`. Este campo es opcional.
`category_id`	ID de la categoría del producto. Es una clave foránea que referencia a la tabla `categories`. Este campo es opcional.
`description`	Descripción detallada del producto. Este campo es opcional y proporciona información adicional sobre el producto.
`available_stock`	Cantidad de unidades disponibles del producto para préstamo. Este campo es obligatorio y se actualiza conforme se prestan los productos.
`is_active`	Indica si el producto está activo y disponible para préstamo. Este campo es booleano y tiene un valor predeterminado de `True`.
`created_at`	Fecha y hora de creación del producto en el sistema. Este campo se asigna automáticamente al momento de la creación del producto.

Cuadro 10: Estructura de la tabla `products`.

La tabla `products` es clave para el manejo eficiente del inventario y la gestión de los préstamos. Permite mantener un registro detallado de todos los productos, su disponibilidad y las características asociadas, y está diseñada para ser fácilmente ampliable en caso de que sea necesario agregar más atributos o relaciones con otras tablas.

5.2.5. Módulo de Dispositivos

Tabla de dispositivos

La tabla `devices` almacena la información detallada de cada elemento de un producto existente en el inventario.

El motivo del nombre **dispositivo** surge a partir de que en el entorno de la tecnología, dentro del inventario del laboratorio, una gran cantidad y variedad de elementos dentro del mismo corresponde a *dispositivos*. Es por ello que, cada dispositivo está asociado con un producto específico y tiene un código interno, número de serie y un estado asociado. Además, idealmente, se puede rastrear su ubicación dentro del sistema.

Campo	**Descripción**
`id`	Identificador único del dispositivo. Es la clave primaria de la tabla.
`product_id`	ID del producto asociado al dispositivo. Es una clave foránea que referencia a la tabla `products`. Este campo es obligatorio.
`internal_code`	Código interno único para identificar el dispositivo. Este campo es opcional.
`serial_number`	Número de serie único del dispositivo. Este campo es opcional y único.
`status_id`	ID del estado del dispositivo. Es una clave foránea que referencia a la tabla `statuses`. Este campo es obligatorio.
`created_at`	Fecha y hora de creación del dispositivo en el sistema. Este campo se asigna automáticamente al momento de la creación.
`ubication_id`	ID de la ubicación del dispositivo. Es una clave foránea que referencia a la tabla `ubications`. Este campo es opcional.

Cuadro 11: Estructura de la tabla `devices`.

Tabla de registros de cambios de estado de dispositivos

La tabla `device_status_logs` se utiliza para registrar cada cambio de estado de los dispositivos en el sistema. Cada vez que se cambia el estado de un dispo-

sitivo, se crea una entrada en esta tabla que incluye el estado anterior, el nuevo estado, la fecha y hora del cambio, el usuario que realizó la modificación y un comentario opcional sobre el cambio.

Campo	**Descripción**
`id`	Identificador único del registro de cambio de estado. Es la clave primaria de la tabla.
`device_id`	ID del dispositivo cuyo estado fue cambiado. Es una clave foránea que referencia a la tabla `devices`. Este campo es obligatorio.
`status_id`	ID del nuevo estado del dispositivo. Es una clave foránea que referencia a la tabla `statuses`. Este campo es obligatorio.
`timestamp`	Fecha y hora en que se registró el cambio de estado. Este campo se asigna automáticamente al momento de la creación del registro.
`user_id`	ID del usuario que realizó el cambio de estado. Es una clave foránea que referencia a la tabla `users`. Este campo es obligatorio.
`comment`	Comentario opcional sobre el cambio de estado, para dar contexto sobre el motivo del cambio. Este campo es opcional.

Cuadro 12: Estructura de la tabla `device_status_logs`.

5.2.6. Módulo de Prestamos

En este módulo, se gestionan las solicitudes de préstamo de dispositivos. Las tablas `device_status_logs` y `statuses`, que ya han sido descritas en las secciones 5.2.5 y 5.2.3, también son utilizadas en este módulo para registrar y gestionar el estado de las solicitudes y los dispositivos a lo largo de su ciclo de vida.

Tabla de solicitudes de préstamo

La tabla `loan_requests` gestiona las solicitudes de préstamo de los usuarios. Cada solicitud incluye información como el usuario que la realizó, el estado de la solicitud, las fechas relacionadas con la entrega y devolución del dispositivo,

así como una razón opcional para el préstamo.

Campo	Descripción
`id`	Identificador único de la solicitud de préstamo. Es la clave primaria de la tabla.
`user_id`	ID del usuario que realiza la solicitud. Es una clave foránea que referencia a la tabla `users`. Este campo es obligatorio.
`request_date`	Fecha y hora en que se realizó la solicitud. Este campo se asigna automáticamente al momento de la creación de la solicitud.
`status_id`	ID del estado de la solicitud. Es una clave foránea que referencia a la tabla `statuses`. Este campo es obligatorio.
`reason`	Motivo opcional de la solicitud de préstamo (por ejemplo, "Necesito el dispositivo para un proyecto").
`delivery_date`	Fecha en que se entrega el dispositivo al usuario. Este campo es opcional.
`estimated_return_date`	Fecha estimada de devolución del dispositivo. Este campo es opcional.
`actual_return_date`	Fecha real de devolución del dispositivo. Este campo es opcional y se completa cuando se retorna el dispositivo.

Cuadro 13: Estructura de la tabla `loan_requests`.

Tabla de elementos de solicitudes de préstamo

La tabla `loan_request_items` gestiona los dispositivos individuales solicitados en una solicitud de préstamo. Cada elemento en esta tabla está asociado con una solicitud específica de préstamo y un dispositivo específico. Esto permite llevar un control detallado de los productos solicitados.

Campo	Descripción
`id`	Identificador único del elemento de la solicitud de préstamo. Es la clave primaria de la tabla.
`loan_request_id`	ID de la solicitud de préstamo asociada. Es una clave foránea que referencia a la tabla `loan_requests`. Este campo es obligatorio.
`device_id`	ID del dispositivo solicitado. Es una clave foránea que referencia a la tabla `devices`. Este campo es obligatorio.

Cuadro 14: Estructura de la tabla `loan_request_items`.

5.3. Modelo de base de datos

En la sección 5.2, se describieron de manera detallada las tablas del sistema, incluyendo sus campos y relaciones, con el objetivo de gestionar de forma eficiente el inventario y el ciclo de vida de los dispositivos. A partir de estas descripciones, el modelo de base de datos se implementa utilizando SQLAlchemy, que actúa como el ORM (Object-Relational Mapping) para facilitar la interacción entre el código Python y la base de datos relacional.

Además, para asegurar la escalabilidad y facilitar futuras modificaciones del esquema, la gestión de las migraciones de la base de datos se realiza con Alembic, una herramienta que permite aplicar cambios incrementales de manera controlada. Esta solución asegura que cualquier modificación en el modelo de datos, ya sea la adición de nuevas tablas o campos, se maneje de manera eficiente y sin afectar los datos existentes.

Todo el proyecto backend está configurado y gestionado dentro de un entorno virtual `.venv`, creado utilizando `uv from astral`. Esto garantiza la separación y aislamiento de dependencias, proporcionando un entorno controlado para el desarrollo y la ejecución del proyecto. La utilización de este entorno virtual facilita la instalación de las bibliotecas necesarias, como SQLAlchemy y Alembic, sin interferir con el sistema global.

5.3.1. Diagrama Entidad Relación

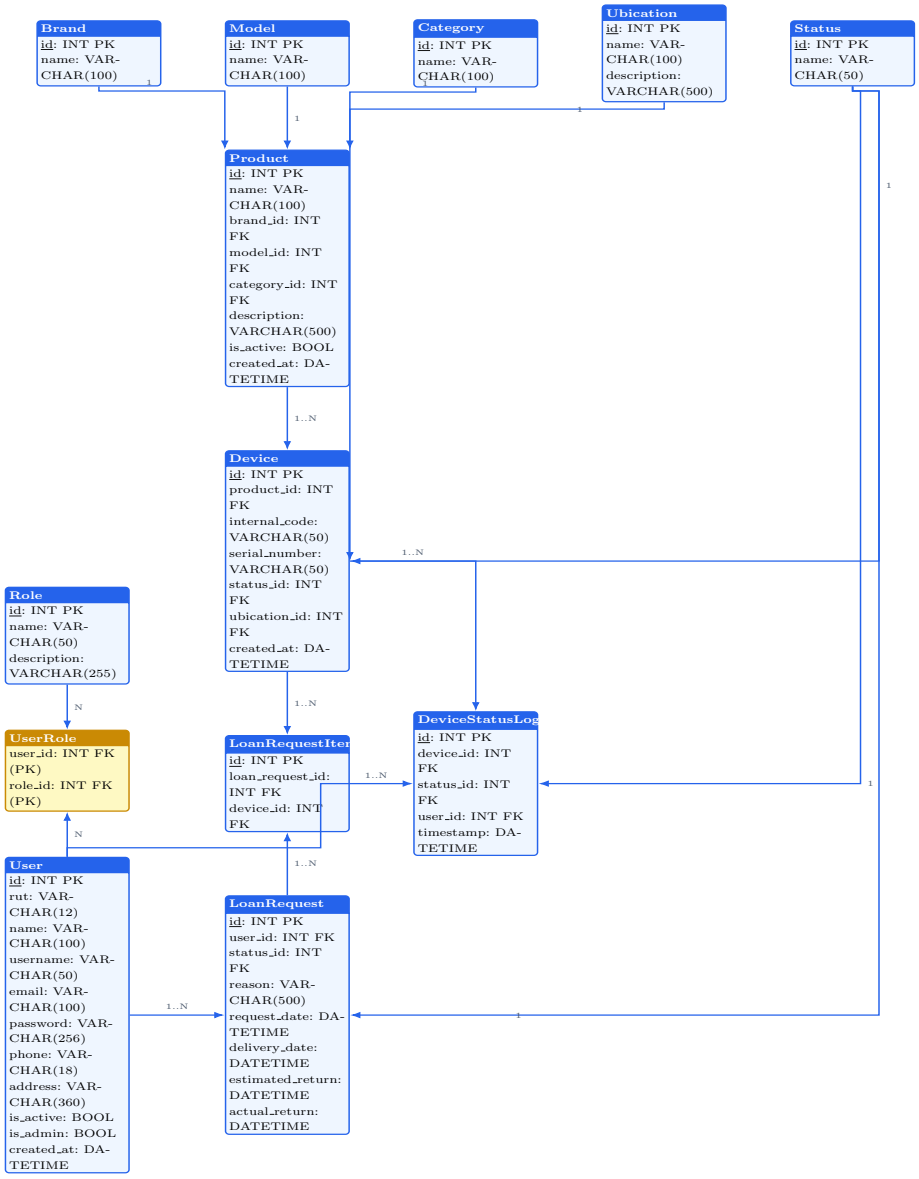

Figura 1: Diagrama Entidad-Relación del modelo de base de datos LabDIC.

6. Arquitectura del sistema

El presente hito documenta las decisiones arquitectónicas que sustentan el desarrollo del sistema, tanto en el backend como en el frontend. Comprender estas decisiones es fundamental para interpretar correctamente el código implementado en los hitos siguientes.

6.1. Arquitectura del backend

DTOs, Repositorios y Controladores

El backend se organiza en torno a tres entidades que dividen responsabilidades de forma clara y evitan que una sola capa concentre demasiada lógica.

Un **DTO** (Data Transfer Object) es un objeto que define la estructura de los datos que viajan entre el backend y el cliente. Su función es actuar como contrato explícito: determina qué campos se exponen en una respuesta, qué campos son obligatorios al crear un recurso y cuáles son opcionales al actualizarlo. En este proyecto los DTOs se implementan como clases `SQLAlchemyDTO` que se vinculan directamente al modelo ORM, con una configuración `SQLAlchemyDTOConfig` que indica qué campos excluir o marcar como opcionales. Esta aproximación evita escribir esquemas de validación duplicados y garantiza coherencia entre el modelo de datos y el contrato de la API.

Un **repositorio** encapsula el acceso a los datos de una tabla. En este proyecto se utiliza `SQLAlchemySyncRepository` de Advanced Alchemy como base, que provee operaciones genéricas ya implementadas: `list`, `get_one`, `add`, `get_and_update` y `delete`. Cuando la lógica de un módulo requiere operaciones que van más allá de estas primitivas, se crean **repositorios personalizados** extendiendo la clase base. La necesidad de personalización surge cuando una operación involucra múltiples tablas en una misma transacción, requiere cargar relaciones mediante `selectinload` para evitar consultas N+1, o debe aplicar lógica de negocio antes de persistir, como el hashing de contraseñas o la actualización coordinada de estados entre tablas vinculadas.

Un **controlador** define los endpoints HTTP del recurso. Hereda de `Controller` de Litestar y declara los métodos decorados con `@get`, `@post`, `@patch` y `@delete`, cada uno correspondiente a una operación CRUD. El controlador no contiene lógica de acceso a datos: su responsabilidad es recibir la petición, inyectar las dependencias necesarias (repositorio, usuario autenticado) y delegar la operación

al repositorio. La transformación de los datos de entrada y salida queda a cargo de los DTOs, que Litestar aplica automáticamente al serializar y deserializar las respuestas.

La delegación entre estas tres entidades sigue siempre el mismo flujo: el **controlador** recibe la petición y valida el acceso; el **DTO** transforma y valida los datos; y el **repositorio** ejecuta la operación sobre la base de datos. Esta separación hace que cada capa sea independientemente legible, testeable y reemplazable.

Estructura de directorios y concepto de servicio

Los módulos del backend se ubican en `app/services/labdic_inventory/`. Cada subdirectorio representa un módulo funcional del sistema y contiene, de forma consistente, los archivos `dtos.py`, `repositories.py` y `controllers.py`. El término *servicio* se utiliza aquí en un sentido arquitectónico amplio: Se refiere al conjunto de componentes (DTO, repositorio y controlador) que implementan la funcionalidad de un módulo. Esta convención de nomenclatura refleja que cada directorio entrega una capacidad completa del sistema, desde el acceso a datos hasta la exposición de la API.

Router del backend

En `app/services/labdic_inventory/router.py` se define `labdic_inventory_router`, una instancia de `Router` de Litestar con el prefijo de ruta `/labdic_inventory`. Este router agrupa todos los controladores del sistema bajo un prefijo común, de modo que los endpoints de cada módulo quedan organizados bajo rutas como `/labdic_inventory/users`, `/labdic_inventory/products`, etcétera. En `app/main.py`, este router se registra como único manejador de rutas de la aplicación Litestar, junto con los plugins de SQLAlchemy y seguridad. El router existe porque centraliza la configuración de rutas y permite aplicar prefijos, middlewares o políticas de seguridad a nivel de grupo sin tener que repetirlos en cada controlador individual.

6.2. Arquitectura del frontend

Router, control de acceso y `RouterView`

El router del frontend se configura en `src/router/index.ts` mediante `createRouter` de Vue Router. Cada ruta asocia una URL a un componente de vista y puede

llevar metadatos como `requiresAuth: true` o `requiresAdmin: true`. El guard global definido en `beforeEach` consulta las stores antes de cada cambio de ruta: si el usuario no está autenticado, redirige al login preservando la URL de destino; si no tiene privilegios de administrador, redirige al home. La directiva `<RouterView />` actúa como punto de montaje dinámico en los layouts: el router sustituye este elemento con el componente de vista que corresponde a la ruta activa, sin recargar el resto del layout.

Stores, Pinia y `localStorage`

Pinia es la librería oficial de gestión de estado global para Vue 3. En este proyecto se utiliza para mantener dos stores persistentes entre recargas de página: `useAuthStore` y `useUserStore`.

`useAuthStore` almacena el token JWT en `localStorage` y expone la propiedad computada `isAuthenticated`. Al iniciar la aplicación, el token se recupera automáticamente del almacenamiento local, lo que permite que el usuario permanezca autenticado sin necesidad de iniciar sesión nuevamente tras una recarga. Cuando el token expira o el usuario cierra sesión, `clearToken` lo elimina tanto del estado reactivo como del `localStorage`.

`useUserStore` almacena el objeto completo del usuario autenticado, incluyendo sus roles. También persiste en `localStorage` para que propiedades como `isAdmin` estén disponibles inmediatamente al recargar, sin necesidad de hacer una petición adicional al backend. Expone propiedades computadas como `isAdmin`, `isUser` y métodos auxiliares como `hasRole` y `hasAnyRole`, que son consumidos tanto por el router como por los componentes de la interfaz para mostrar u ocultar elementos según el rol del usuario.

El uso de `localStorage` en lugar de cookies de sesión es una decisión deliberada: simplifica la implementación en un entorno de desarrollo local y es coherente con el flujo de autenticación basado en Bearer tokens de la API.

Carpeta `types/` y contrato con la API

La carpeta `src/types/` contiene las interfaces TypeScript que definen la estructura de los datos que el frontend espera recibir o enviar a la API. Estas interfaces son el equivalente frontend de los DTOs del backend: establecen un contrato de tipos que el compilador puede verificar en tiempo de desarrollo, reduciendo errores de integración. Por ejemplo, `device.types.ts` define la interfaz

Device con todos los campos que expone DeviceReadDTO, y DevicePayload con los campos requeridos para crear o actualizar un dispositivo.

Un aspecto importante es la conversión de nomenclatura: el backend retorna JSON con claves en snake_case (convención de Python), mientras que el frontend trabaja en camelCase (convención de TypeScript). La función apiFetch en src/services/api.ts aplica esta conversión automáticamente en ambas direcciones usando la librería change-case/keys: convierte las claves del cuerpo enviado a snake_case antes de hacer la petición, y convierte las claves de la respuesta recibida a camelCase antes de retornarla. Esto permite que las interfaces en types/ usen camelCase de forma consistente sin que los componentes deban preocuparse por la convención del backend.

Carpeta services/ y función apiFetch

La carpeta src/services/ centraliza las funciones que realizan peticiones HTTP a la API. Cada archivo corresponde a un módulo del sistema (user.service.ts, device.service.ts, loan.service.ts, etcétera) y exporta funciones con nombres descriptivos como getDevices, createLoan o deleteProduct. Todas estas funciones delegan en apiFetch, la función central definida en src/services/api.ts.

apiFetch cumple varias responsabilidades de forma transversal: adjunta automáticamente el header Authorization: Bearer {token} si el usuario está autenticado, serializa el cuerpo de la petición a JSON con conversión de claves a snake_case, deserializa la respuesta con conversión a camelCase, maneja el código de respuesta 204 No Content para operaciones de eliminación, y ante un error 401 limpia las stores y redirige al login indicando que la sesión expiró. Esta centralización evita repetir la lógica de autenticación y manejo de errores en cada llamada individual, y garantiza un comportamiento consistente en toda la aplicación.

La razón de tener una carpeta services/ separada de las vistas es de separación de responsabilidades: los componentes Vue no deben conocer los detalles del protocolo HTTP ni la estructura exacta de las URLs. Al delegar en los servicios, los componentes solo necesitan llamar a getDevices() o createLoan(payload) y recibir datos ya tipados, sin acoplarse a la infraestructura de comunicación.

6.3. Flujo de consumo de la API

El flujo completo desde la base de datos hasta la interfaz de usuario puede describirse en cuatro capas que se articulan de forma secuencial.

Capa de datos. PostgreSQL actúa como motor relacional. SQLAlchemy mapea las tablas a clases Python mediante el ORM declarativo, y `database.py` conecta PostgreSQL con Litestar a través de `SQLAlchemySyncConfig`, que recibe la cadena de conexión `postgresql+psycopg2:///labdic_inventory` desde la configuración del proyecto y registra el plugin de SQLAlchemy en la aplicación. Alembic se integra con el mismo modelo de SQLAlchemy para gestionar las migraciones versionadas del esquema, manteniendo la base de datos sincronizada con el código a lo largo del desarrollo.

Capa de API. Litestar expone los datos a través de la API REST. `SQLAlchemySyncRepository` provee el acceso a datos desde los controladores, y `SQLAlchemyDTO` transforma los modelos ORM en respuestas JSON con los campos adecuados según el contexto. La seguridad se gestiona en `security.py` mediante `OAuth2PasswordBearerAuth`, que intercepta cada petición, valida el token JWT y resuelve el usuario autenticado antes de que el controlador ejecute la operación solicitada. Todas las rutas quedan agrupadas bajo el prefijo `/labdic_inventory` a través del router del backend.

Puente frontend-backend. La carpeta `types/` define las interfaces TypeScript que representan los DTOs del backend en el lado del cliente, estableciendo el contrato de tipos. La carpeta `services/` implementa las funciones que consumen la API usando `apiFetch`, que se encarga de adjuntar el token de autenticación, convertir las claves entre `snake_case` y `camelCase`, y manejar los errores de forma centralizada. La URL base del backend (`http://localhost:8000`) se configura en `api.ts`, y el backend permite las peticiones del frontend mediante la configuración de CORS en `main.py`.

Capa de presentación. En el frontend, Pinia gestiona el estado global persistido en `localStorage`: el token JWT en `useAuthStore` y los datos del usuario autenticado en `useUserStore`. Vue Router controla la navegación y el acceso a las rutas usando la información de las stores. Los módulos del frontend, organizados por funcionalidad, consumen los servicios para obtener datos y los presentan a través de componentes de PrimeVue y TailwindCSS, garantizando una interfaz coherente y adaptable.

El diagrama de la figura 2 sintetiza este flujo de forma visual.

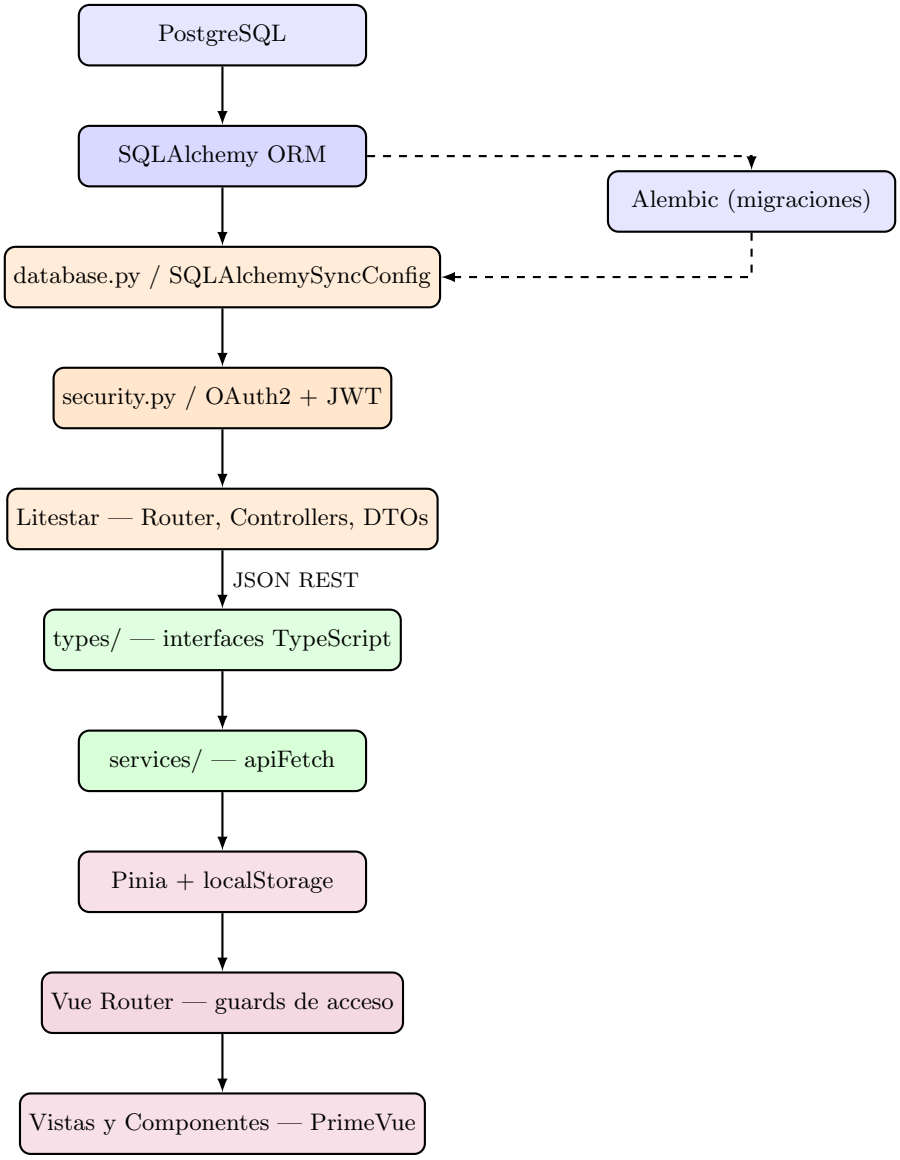

Figura 2: Flujo de consumo de la API del sistema LabDIC.

7. Implementación de base de datos y backend del sistema

7.1. Uso de Alembic a lo largo del desarrollo

El desarrollo del backend no siguió un esquema rígido en el que el modelo de base de datos se diseña una sola vez y permanece intacto. Por el contrario, a medida que se avanzó en la implementación de los servicios y, más tarde, en la construcción del frontend, surgieron necesidades que requirieron ajustes en el esquema. Alembic fue la herramienta que permitió gestionar estos cambios de forma controlada y trazable, generando un script de migración por cada modificación relevante.

A lo largo del proyecto se generaron un total de 8 migraciones versionadas, nombradas bajo la convención `Parche X.Y`, que documentan la evolución del esquema desde su creación hasta su forma final. La migración inicial estableció todas las tablas del sistema. Las siguientes migraciones atendieron ajustes puntuales que surgieron durante el desarrollo: la ampliación del campo `password` para compatibilidad con `pwdlib`, la eliminación del campo `comment` de `device_status_logs` al redefinir la responsabilidad del módulo, y el registro explícito de `UserRole` como modelo ORM para que SQLAlchemy gestionara correctamente la tabla intermedia. Finalmente, los parches 1.5 a 1.8 resolvieron una serie de problemas con la propagación de eliminaciones en cascada (`ON DELETE CASCADE`), detectados al implementar el frontend: al eliminar un `product`, sus `devices` no se eliminaban; y al eliminar un `device`, sus `loan_request_items` quedaban huérfanos. Las relaciones corregidas fueron `devices` → `products`, `loan_request_items` → `devices`, y `loan_request_items` → `loan_requests`.

Este historial de migraciones ilustra de forma concreta cómo Alembic actúa como red de seguridad durante el desarrollo iterativo: cada cambio queda documentado, es reversible mediante `downgrade`, y puede aplicarse en cualquier entorno con un único comando, garantizando coherencia entre el modelo Python y el esquema real de la base de datos.

7.2. Módulo de Usuarios

El módulo de `users` concentra la gestión de identidad y datos personales de quienes interactúan con el sistema. Su implementación consta de un repositorio, un conjunto de DTOs y un controlador que expone la API REST.

DTOs

Se definen tres DTOs para el modelo `User`:

- `UserCreateDTO`: utilizado al crear un usuario; excluye `id` y `created_at`, que se generan automáticamente.
- `UserReadDTO`: utilizado en todas las respuestas; excluye `password` para evitar exponer credenciales.
- `UserUpdateDTO`: utilizado en actualizaciones parciales; todos los campos son opcionales (`partial=True`).
- `UserLoginDTO`: utilizado exclusivamente por el servicio `auth` para el flujo de login; expone únicamente `username` y `password`.

Repositorio

`UserRepository` extiende `SQLAlchemySyncRepository` e incorpora tres métodos personalizados:

- `get_my_user(username)`: recupera un usuario por su `username` con todas sus relaciones cargadas mediante `selectinload` (`roles`, `loan_requests`, `status_logs`). Es utilizado por el sistema de autenticación para resolver el usuario a partir del token JWT.
- `add_with_existing_roles(roles_repo, user)`: crea un usuario hasheando su contraseña con `pwdlib` y asociándolo a roles ya existentes en la base de datos, consultados a través de `RoleRepository`. Este método personalizado es necesario porque la creación de un usuario involucra dos tablas (`users` y `user_roles`) y requiere cifrar la contraseña antes de persistir.
- `update_with_existing_roles(roles_repo, user_id, user_data)`: actualiza los campos de un usuario y, si el payload incluye roles, resuelve los objetos `Role` correspondientes antes de aplicar el cambio. Permite actualizaciones parciales respetando el contrato de `UserUpdateDTO`.
- `check_password(username, password)`: verifica la contraseña de un usuario comparando el texto plano contra el hash almacenado mediante `pwdlib`. Utilizado por el controlador de `auth` durante el login.

Los métodos genéricos `list`, `get_one` y `delete` provienen directamente de `SQLAlchemySyncRepository` y no requieren implementación adicional.

Controlador y guards

`UserController` expone la ruta base `/users` e implementa los siguientes endpoints:

- `GET /users/me`: retorna el usuario autenticado extrayéndolo del objeto `request`. No requiere parámetros adicionales.
- `GET /users/`: lista todos los usuarios mediante `list()`.
- `GET /users/{user_id}`: obtiene un usuario por ID mediante `get_one()`; retorna `404` si no existe.
- `POST /users/`: crea un usuario con roles; protegido por `admin_user_guard`. Delega en `add_with_existing_roles`.
- `PATCH /users/{user_id}`: actualiza un usuario; protegido por `admin_user_guard`. Delega en `update_with_existing_roles`.
- `DELETE /users/{user_id}`: elimina un usuario; protegido por `admin_user_guard`.

Se definen dos guards en este módulo:

- `require_active`: aplicado a nivel de controlador; rechaza con `403` cualquier petición de un usuario cuya cuenta esté inactiva (`is_active = False`). Garantiza que ningún usuario deshabilitado pueda operar el sistema.
- `admin_user_guard`: aplicado a los endpoints de escritura (`POST`, `PATCH`, `DELETE`); rechaza con `403` si el usuario autenticado no tiene `is_admin = True`. Ambas guards reciben `ASGIConnection` para acceder al usuario ya resuelto por el middleware JWT.

7.3. Módulo de Autenticación

La autenticación se implementa a través del servicio `auth`, cuyo controlador `AuthController` expone un único endpoint:

- `POST /auth/login`: recibe las credenciales del usuario codificadas como `application/x-www-form-urlencoded` mediante `UserLoginDTO`. Verifica la existencia del usuario con `get_one_or_none` y la validez de la contraseña con `check_password`; si alguna falla, responde con `401`. En caso de éxito, delega en `oauth2_auth.login` para emitir el token JWT.

La configuración de seguridad centralizada reside en `security.py`, donde se instancia `OAuth2PasswordBearerAuth` con una función `retrieve_user_handler` que, dado un token, abre sesión en la base de datos y recupera el usuario mediante `get_my_user`. Este objeto `oauth2_auth` se registra como middleware de la aplicación Litestar, de modo que todas las rutas protegidas reciben automáticamente el usuario autenticado en `request.user`, disponible también en las guards a través de `ASGIConnection`.

7.4. Módulo de Catálogo

El catálogo agrupa cinco entidades de referencia: `brands`, `models`, `categories`, `statuses` y `ubications`, además de `roles`, que fue integrado aquí por compartir la misma naturaleza de dato de referencia con métodos acotados. Todos estos recursos siguen una estructura homogénea y predecible.

Cada entidad del catálogo posee su propio par de DTOs (`ReadDTO`, `CreateDTO`, `UpdateDTO`), su repositorio basado en `SQLAlchemySyncRepository`, y un controlador con los cuatro endpoints genéricos: `GET /`, `GET /{id}`, `POST /`, `PATCH /{id}` y `DELETE /{id}`. Los DTOs excluyen siempre las relaciones inversas para evitar serialización circular (por ejemplo, `BrandReadDTO` excluye `products`).

Todos los métodos utilizados son genéricos y provienen directamente de `SQLAlchemySyncRepository`: `list`, `get_one`, `add`, `get_and_update` y `delete`. No se implementó ningún método personalizado en este módulo, ya que la simplicidad de sus entidades no lo requiere.

Una excepción destacable es el controlador de `roles`, que define la constante `PROTECTED_ROLES = {'usuario'}` y verifica en el endpoint `DELETE` que el rol a eliminar no pertenezca a dicho conjunto. De ser así, responde con `409 Conflict` para proteger roles del sistema que son necesarios para el funcionamiento correcto de la plataforma.

7.5. Módulo de Productos

El módulo de `products` gestiona los tipos de ítems disponibles en el inventario. Su estructura sigue el patrón estándar del proyecto con tres DTOs, un repositorio y un controlador.

DTOs

- `ProductReadDTO`: excluye `devices` para evitar serialización de todas las unidades físicas al listar productos.
- `ProductCreateDTO`: excluye `id`, `created_at` y `devices`.
- `ProductUpdateDTO`: igual que `CreateDTO` pero con `partial=True`.

Repositorio y método personalizado de actualización

`ProductRepository` utiliza exclusivamente métodos genéricos para `list`, `get_one` y `delete`. Sin embargo, el endpoint de actualización implementa lógica personalizada directamente en el controlador, dado que `as_builtins()` con `partial=True` omite los campos con valor `None`, lo que impediría limpiar relaciones opcionales como `brand_id`, `model_id` o `category_id`. Para resolverlo, el controlador aplica estos tres campos directamente sobre la instancia con `setattr`, forzando a SQLAlchemy a persistir el valor `None` de forma explícita.

Controlador

Expone `GET /products/`, `GET /products/{id}`, `POST /products/`, `PATCH /products/{id}` y `DELETE /products/{id}` sin restricciones adicionales de guards en este módulo.

7.6. Módulo de Dispositivos

El módulo de `devices` administra las unidades físicas del inventario, su estado actual y el historial de cambios de estado. Es el módulo con mayor cantidad de métodos personalizados del proyecto.

DTOs

- `DeviceReadDTO`: excluye `loan_request_items` y `status_logs`.
- `DeviceCreateDTO`: excluye `id`, `created_at`, relaciones inversas y objetos de relación (`product`, `status`, `ubication`).
- `DeviceUpdateDTO`: igual que `CreateDTO` más `status_id`, ya que el estado se gestiona por un endpoint dedicado.

- `DeviceStatusChangeDTO`: dataclass simple con un único campo `status_id`, utilizado exclusivamente por el endpoint de cambio de estado.
- `DeviceStatusLogReadDTO`: expone el historial de estados excluyendo la relación inversa con `device`.

Repositorio

`DeviceRepository` define los siguientes métodos personalizados, además de los genéricos heredados:

- `list_all_with_relations()`: lista todos los dispositivos con `product`, `status` y `ubication` cargados mediante `selectinload`, evitando consultas N+1 al serializar.
- `list_by_product(product_id)`: filtra dispositivos por producto, útil para mostrar las unidades físicas de un ítem específico en el frontend.
- `list_available()`: filtra dispositivos cuyo `status.name == "disponible"` mediante un `JOIN`, exponiendo solo los ítems prestables al momento de crear una solicitud.
- `get_with_relations(device_id)`: obtiene un dispositivo por ID con todas sus relaciones cargadas.
- `change_status(device_id, status_id, user_id)`: actualiza el `status_id` del dispositivo y genera automáticamente un registro en `device_status_logs` con el usuario que realizó el cambio y la marca temporal. Este método es el **núcleo del historial** del inventario.

`DeviceStatusLogRepository` implementa el método `get_history_by_device(device_id)`, que retorna el historial de estados de un dispositivo ordenado cronológicamente.

Controlador

`DeviceController` expone los siguientes endpoints:

- `GET /devices/`: lista todos los dispositivos; acepta el parámetro de query opcional `?product_id=` para filtrar por producto, delegando en `list_by_product` o `list_all_with_relations` según corresponda.

- `GET /devices/available`: lista solo los dispositivos disponibles para préstamo.
- `GET /devices/{device_id}`: obtiene un dispositivo por ID con sus relaciones.
- `POST /devices/`: registra un nuevo dispositivo en el inventario.
- `PATCH /devices/{device_id}`: actualiza campos generales del dispositivo, excluyendo el estado.
- `PATCH /devices/{device_id}/status`: cambia el estado del dispositivo y registra el cambio en el historial. Requiere el usuario autenticado (`request.user.id`) para atribuir el cambio.
- `GET /devices/{device_id}/history`: retorna el historial de cambios de estado del dispositivo.
- `DELETE /devices/{device_id}`: elimina un dispositivo; gracias al `ON DELETE CASCADE` definido en las migraciones, sus `loan_request_items` asociados se eliminan en cascada.

7.7. Módulo de Préstamos

El módulo de `loans` modela el ciclo de vida completo de las solicitudes de préstamo. Es el módulo de mayor complejidad funcional, pues sus operaciones involucran múltiples tablas y actualizaciones de estado coordinadas.

DTOs

`LoanRequestReadDTO` expone la solicitud con sus relaciones anidadas: `user`, `status`, `loan_request_items` con sus `device` y el `product` de cada uno. Dado que estas relaciones son bidireccionales en el modelo ORM, sin una exclusión explícita el serializador intentaría seguir las referencias en ambas direcciones indefinidamente: desde `LoanRequest` hacia `Device`, y desde `Device` de vuelta hacia `LoanRequest` a través de `loan_request_items`, generando un ciclo infinito que termina en error. Por ello, el DTO excluye explícitamente campos como `loan_request_items.loan_request`, `loan_request_items.device.loan_request_items` y otros que cierran el ciclo, además de ocultar datos sensibles como `user.password`.

LoanRequestCreateDTO, en cambio, se implementa como un dataclass en lugar de un SQLAlchemyDTO. Esto se debe a que el payload de creación no corresponde directamente a la estructura de ninguna tabla: el cliente envía una lista de device_ids, un reason y una estimated_return_date, mientras que el repositorio se encarga de construir internamente tanto la LoanRequest como los LoanRequestItem asociados. Forzar este payload dentro de un SQLAlchemyDTO implicaría exponer la estructura interna del modelo ORM al cliente, acoplando innecesariamente la API al esquema de la base de datos [46]. El dataclass permite definir un contrato de entrada limpio, independiente del modelo, que el repositorio traduce a las operaciones de persistencia correspondientes. [47]

Repositorio

LoanRequestRepository implementa los siguientes métodos personalizados:

- list_with_relations(): lista todas las solicitudes con sus relaciones cargadas, ordenadas por id descendente. **Utilizado por administradores**.
- list_mine(user_id): filtra las solicitudes del usuario autenticado. Permite que cada usuario vea solo sus propios préstamos.
- get_with_relations(loan_id): obtiene una solicitud por ID con todas sus relaciones.
- create_with_items(user_id, device_ids, reason, estimated_return_date): crea una solicitud en estado 'pendiente' y asocia en una sola transacción los LoanRequestItem correspondientes a cada device_id recibido. Utiliza session.flush() para obtener el id de la solicitud antes del commit.
- approve(loan_id): cambia el estado de la solicitud a 'aprobado'.
- reject(loan_id): cambia el estado a 'rechazado'.
- deliver(loan_id): marca la solicitud como 'prestado', registra la delivery_date y actualiza el status_id de cada dispositivo asociado a 'prestado'.
- register_return(loan_id): marca la solicitud como 'devuelto', registra la actual_return_date y restaura el status_id de cada dispositivo a 'disponible'.

Controlador

`LoanRequestController` expone los siguientes endpoints:

- `GET /loans/`: lista todas las solicitudes; protegido por `admin_guard`, solo accesible para administradores.
- `GET /loans/me`: lista las solicitudes del usuario autenticado.
- `GET /loans/{loan_id}`: detalle de una solicitud con sus ítems y dispositivos.
- `POST /loans/`: crea una solicitud de préstamo; el `user_id` se extrae del token (`request.user.id`), no del payload, para garantizar que el usuario no pueda crear solicitudes en nombre de otros.
- `PATCH /loans/{loan_id}/approve`: aprueba una solicitud; protegido por `admin_guard`.
- `PATCH /loans/{loan_id}/reject`: rechaza una solicitud; protegido por `admin_guard`.
- `PATCH /loans/{loan_id}/deliver`: registra la entrega; protegido por `admin_guard`.
- `PATCH /loans/{loan_id}/return`: registra la devolución; protegido por `admin_guard`.
- `DELETE /loans/{loan_id}`: elimina una solicitud; gracias al `ON DELETE CASCADE`, sus `loan_request_items` se eliminan en cascada.

8. Implementación del frontend del sistema

El desarrollo del frontend se organizó en seis fases progresivas, donde cada una deja las condiciones preparadas para que la siguiente pueda avanzar sin fricciones. Esta estrategia permitió mantener en todo momento una aplicación funcional y coherente, evitando estados intermedios rotos o incompletos.

8.1. Fase 1: Módulo de Usuarios y Autenticación

La primera fase establece las bases del sistema desde la perspectiva del usuario final: el acceso a la aplicación y la navegación según el rol asignado.

Layouts

Se definen dos layouts que encapsulan la estructura visual de las rutas de la aplicación. `AuthLayout` es el layout público, utilizado exclusivamente por `/login`; centra el contenido sobre un fondo degradado oscuro y no incluye ningún elemento de navegación, como se aprecia en la figura 4. `AppLayout` es el layout principal para todas las rutas privadas; integra `AppSidebar` y `AppTopbar` como componentes de navegación persistentes, mientras que el contenido de cada ruta se renderiza dentro de `<RouterView />`. La separación entre ambos layouts permite que la pantalla de login sea completamente independiente del shell de la aplicación, sin necesidad de condicionar su visibilidad desde un único componente raíz.

LoginView y flujo de autenticación

`LoginView` recibe las credenciales del usuario y delega en el servicio `auth.service` para realizar el `POST /auth/login`. Ante una respuesta exitosa, el token JWT se persiste en `localStorage` a través de `useAuthStore`, y los datos del usuario autenticado se cargan inmediatamente mediante `getMyUser()` y se almacenan en `useUserStore`. Si el usuario provino de una ruta protegida, el router lo redirige al destino original; en caso contrario, a `HomeView`, cuya vista se puede apreciar en las figuras 5 y 6. Los errores de credenciales (`401`) y de cuenta inactiva (`403`) se muestran como mensajes descriptivos en la misma vista, sin requerir componentes externos.

Control de acceso y guards de navegación

El control de acceso se implementa en `router/index.ts` mediante un guard global ejecutado en `beforeEach`. Las rutas privadas llevan la meta `requiresAuth: true`; las rutas exclusivas de administrador llevan además `requiresAdmin: true`. El guard consulta `useAuthStore` para verificar autenticación y `useUserStore` para verificar el rol de administrador, redirigiendo al login o al home según corresponda. Esto garantiza que un usuario con `is_admin = false` solo pueda acceder a `HomeView`, `DeviceCatalogView`, `MyLoansView` y `MyUserView`. Dentro de las figuras 5 y 6 se aprecia el control de acceso diferenciado para cada tipo de usuario.

Vistas de usuario

`UsersListView` es la vista de administración de usuarios, accesible solo bajo `/admin/users`; permite listar, crear, editar y eliminar usuarios, y gestionar sus roles mediante el repositorio de `roles`, como se aprecia en la figura 7. `MyUserView` expone los datos del usuario autenticado obtenidos desde `GET /users/me`, permitiendo consultar su perfil sin necesidad de conocer su identificador numérico, como se aprecia en la figura 8.

8.2. Fase 2: Módulo de Catálogo

La segunda fase implementa la gestión del catálogo de referencia, exclusiva para administradores.

`CatalogView`, disponible en `/admin/catalog`, agrupa en una sola vista la administración de `brands`, `models`, `categories`, `statuses` y `ubications`, algunas de cuyas secciones se pueden apreciar en las figuras 9, 10, 11 y 12. Cada entidad se presenta en una sección con su propia tabla y formulario de creación y edición. Las operaciones disponibles son listado, creación, edición y eliminación, todas consumiendo los endpoints del módulo de catálogo del backend.

Una consideración importante respecto a `statuses` (ver figura 11): los estados del sistema son estáticos e imborrables desde la interfaz. Esto se debe a que su uso es único y vital para el funcionamiento del ciclo de vida de los préstamos y del historial de dispositivos: nombres como `'disponible'`, `'pendiente'`, `'aprobado'`, `'rechazado'`, `'prestado'` y `'devuelto'` son referenciados por nombre directamente en la lógica del repositorio del backend. Eliminar o renombrar un estado desde el frontend comprometería el funcionamiento de todo el sistema, por lo que la vista solo permite su visualización, sin exponer botones de edición ni eliminación para esta entidad.

8.3. Fase 3: Módulo de Productos

La tercera fase introduce la primera vista con distinción explícita de roles dentro del mismo recurso.

`ProductsListView`, accesible en `/admin/products`, permite al administrador crear, editar y eliminar productos. Cada producto puede vincularse opcionalmente a entidades del catálogo: marca, modelo y categoría. `ProductDetailView`, accesible en `/admin/products/:id`, expone el detalle completo de un producto junto con los dispositivos asociados a él. Las vistas implementadas para

ProductsListView se pueden apreciar en las figuras 13, 14 y 15.

Es a partir de esta fase donde comienza a evidenciarse en el frontend la necesidad de las eliminaciones en cascada en la base de datos. Al implementar la eliminación de un producto desde la interfaz, se detectó que los dispositivos asociados a dicho producto no se eliminaban correctamente, dejando registros huérfanos. Esto requirió la corrección de la relación ON DELETE CASCADE entre devices y products, gestionada mediante Alembic tal como se describe en la sección 7.1. Esta fase dejó los productos con su estructura de datos completa y sus relaciones con el catálogo correctamente establecidas, listos para ser consumidos por el módulo de dispositivos en la fase siguiente.

8.4. Fase 4: Módulo de Dispositivos

La cuarta fase implementa la gestión de las unidades físicas del inventario y habilita la interacción del usuario común con el catálogo de dispositivos disponibles.

DevicesListView, accesible en /admin/devices, permite al administrador registrar nuevos dispositivos, editarlos, cambiar su estado y eliminarlos. Cada dispositivo se vincula a un producto y, opcionalmente, a una ubicación del catálogo, como se aprecia en las figuras 16 y 17. Al igual que en la fase anterior, la eliminación de un dispositivo requirió la corrección de la relación ON DELETE CASCADE entre loan_request_items y devices, también gestionada con Alembic según lo descrito en la sección 7.1. Adicionalmente, la figura 18 ilustra la integración de un botón dentro de ProductsListView que permite agregar dispositivos directamente desde la vista, realizando un envío hacia DevicesListView con el filtro del producto seleccionado aplicado.

DeviceCatalogView, accesible en /catalog para todos los usuarios autenticados, es la vista principal de interacción para el usuario común. Muestra únicamente los dispositivos con estado "disponible", consumiendo el endpoint GET /devices/available. La vista incluye filtros por categoría y búsqueda por texto, y permite seleccionar uno o más dispositivos mediante casillas de verificación, como se aprecia en la figura 19. Al confirmar la selección, se abre un diálogo donde el usuario puede ingresar un motivo y una fecha estimada de devolución, dejando todo listo para enviar la solicitud de préstamo, como se muestra en la figura 20. La lógica de creación del préstamo, sin embargo, no se activa en esta fase: los formularios y la selección quedan preparados para ser conectados al módulo de préstamos en la fase siguiente. Adicionalmente, la

vista verifica solicitudes activas del usuario antes de abrir el diálogo, evitando solicitudes duplicadas para un mismo dispositivo.

8.5. Fase 5: Módulo de Préstamos

La quinta fase conecta toda la lógica preparada en fases anteriores para implementar el ciclo de vida completo de los préstamos.

Ciclo de vida de un préstamo

Un préstamo atraviesa un conjunto ordenado de estados desde su creación hasta su cierre. El diagrama siguiente ilustra este ciclo, incluyendo el camino nominal y la rama de rechazo:

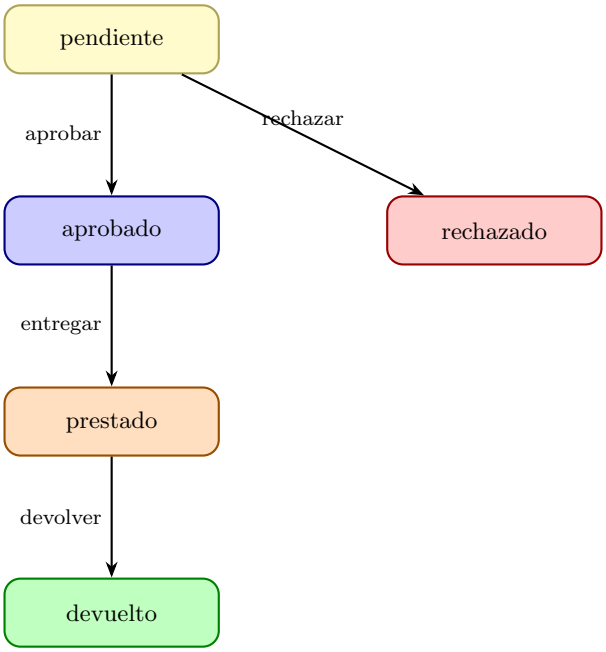

Figura 3: Ciclo de vida de una solicitud de préstamo en LabDIC.

Cada transición corresponde a una acción exclusiva del administrador y es irreversible: una solicitud aprobada no puede volver a estado pendiente, ni una solicitud rechazada puede retomarse. Los colores reflejan la semántica del estado: amarillo para pendiente (en espera de decisión), azul para aprobado (autorizado pero no entregado), rojo para rechazado (ciclo cerrado sin entrega), naranja

para prestado (dispositivos en posesión del usuario) y verde para devuelto (ciclo cerrado exitosamente).

Vistas implementadas

`AdminLoansView`, accesible en `/admin/loans`, permite al administrador visualizar todas las solicitudes del sistema en una tabla con `LoanStatusBadge` para identificar rápidamente el estado de cada una. Al seleccionar una solicitud se abre un `Drawer` lateral con el detalle completo: datos del solicitante, fechas del ciclo, dispositivos involucrados y un timeline visual del ciclo de vida. Desde este mismo drawer el administrador puede ejecutar las acciones disponibles según el estado actual: aprobar o rechazar si está pendiente, registrar entrega si está aprobada, y registrar devolución si está prestada. Las vistas de `AdminLoansView` se aprecian en las figuras 21, 22 y 23.

`MyLoansView`, accesible en `/my-loans` para todos los usuarios autenticados, muestra las solicitudes propias del usuario con el mismo componente de timeline, permitiendo hacer seguimiento del estado de cada préstamo sin necesidad de contactar al administrador, como se aprecia en las figuras 24 y 25.

8.6. Fase 6: UX, UI

Revisión de experiencia de usuario

Una vez implementadas todas las funcionalidades del sistema, se realizó una revisión iterativa de la experiencia de usuario desde la perspectiva del desarrollador como usuario de prueba. Este proceso consistió en recorrer todos los flujos de la aplicación, tanto desde el rol de usuario común como desde el rol de administrador, con el objetivo de identificar inconsistencias visuales, comportamientos inesperados y oportunidades de mejora que no son evidentes durante la implementación técnica.

Entre los ajustes realizados se incluyen: refinamiento de colores en los `LoanStatusBadge` y `DeviceStatusBadge` para mejorar la legibilidad según el estado; ajuste de fondos y tipografías para mantener coherencia visual entre modo claro y modo oscuro, ambos soportados de forma nativa a través del sistema de temas de PrimeVue y TailwindCSS; incorporación de mensajes `Toast` en operaciones críticas (creación, edición, eliminación, errores de red, sesión expirada, solicitudes duplicadas) para dar retroalimentación inmediata al usuario sin interrumpir el flujo de navegación; y validaciones visuales en formularios para prevenir el envío de

datos incompletos o inválidos.

Adicionalmente, `HomeView` fue enriquecida con métricas simples que ofrecen al usuario una visión inmediata del estado del sistema al iniciar sesión. Según el rol del usuario autenticado, la vista muestra tarjetas de resumen con información relevante —como el número de dispositivos disponibles, solicitudes activas o préstamos pendientes de gestión— cada una con acceso directo a su vista correspondiente. Esto reduce la necesidad de navegar manualmente por el menú para obtener un panorama general, mejorando la eficiencia del flujo de trabajo tanto para administradores como para usuarios comunes, como se aprecia en las figuras 5 y 6.

La adaptación *responsive* se verificó probando la aplicación en distintos tamaños de ventana, en orientación horizontal y vertical, y en diferentes navegadores (Firefox, Chrome y Edge). El layout basado en `AppSidebar` colapsable y el uso de componentes de PrimeVue con soporte nativo para pantallas pequeñas permitió que la interfaz se adapte de forma satisfactoria tanto a resoluciones de escritorio como a resolución adaptada a dispositivos móviles con el modo de orientación vertical en los navegadores de escritorio.

Evidencias de la implementación

Las evidencias visuales de las 6 fases de desarrollo e implementación del frontend se encuentran consolidadas en el anexo A, donde se presentan todas las pantallas y vistas funcionando tal como se describen en las secciones 8.1 a 8.6. Con esto concluye la implementación del sistema, el cual, al ser modular y escalable, abre paso a mejoras y extensiones futuras descritas en la sección 10.

9. Entorno de ejecución

Tecnologías utilizadas

La plataforma LabDIC está compuesta por dos servicios independientes: un backend desarrollado en Python y un frontend desarrollado en TypeScript con Vue 3. La tabla 15 resume las tecnologías y versiones utilizadas.

Componente	Tecnología	Versión
Base de datos	PostgreSQL	16
ORM / Migraciones	SQLAlchemy + Alembic	—
Backend	Python + Litestar	3.13
Gestor de entorno backend	uv	0.6
Frontend	Vue 3 + TypeScript + Vite	—
Librería de componentes UI	PrimeVue + TailwindCSS	—
Gestor de entorno frontend	Bun	1.2

Cuadro 15: Tecnologías y versiones del proyecto LabDIC.

Requisitos previos

Para levantar la plataforma de forma local se requiere tener instalado:

- **PostgreSQL 16**: motor de base de datos relacional.
- **Python 3.13**: lenguaje del backend.
- **uv 0.6**: gestor de entornos y dependencias Python (`https://docs.astral.sh/uv/`).
- **Bun 1.2**: gestor de paquetes y entorno de ejecución para el frontend (`https://bun.sh/`).

Variables de entorno

Cada servicio requiere un archivo `.env` con su configuración. Los valores indicados corresponden al entorno de desarrollo local.

Backend (`app-backend/.env`):

```
DATABASE_URL=postgresql+psycopg2:///labdic_inventory
CORS_ALLOWED_ORIGINS=["http://localhost:5173"]
```

Frontend (`app-frontend/.env`):

```
VITE_API_URL=http://localhost:8000
```

Preparación de la base de datos

Antes de levantar el backend es necesario crear la base de datos en PostgreSQL:

```
createdb labdic_inventory
```

Si se utiliza un usuario o contraseña específicos, la variable `DATABASE_URL` debe ajustarse con el siguiente formato:

```
postgresql+psycopg2://usuario:contrasena@localhost:5432/labdic_inventory
```

Levantar el backend

Desde el directorio `app-backend`, ejecutar los siguientes comandos en orden:

```
cd app-backend
uv sync                        # Instala dependencias
uv run alembic upgrade head    # Aplica migraciones a la DB
uv run python seed.py          # Puebla datos base
uv run litestar run --reload   # Levanta la API en modo
    desarrollo
```

Levantar el frontend

En otra terminal, desde el directorio `app-frontend`:

```
cd app-frontend
bun install      # Instala dependencias
bun run dev      # Levanta el servidor de desarrollo
```

URLs de acceso

Una vez levantados ambos servicios, la plataforma queda accesible en las siguientes direcciones:

Servicio	URL
Frontend	`http://localhost:5173`
Backend	`http://localhost:8000`
API Docs	`http://localhost:8000/schema/swagger`

9.1. Contenedorización con Docker

configuración y consideraciones para despliegue

Si bien la plataforma se desarrolló y probó en un entorno local, se dejó preparada una configuración de Docker Compose que permite levantar la aplicación completa (base de datos, backend y frontend) en contenedores aislados. Esta sección documenta la configuración realizada, las decisiones tomadas y los conflictos encontrados durante el proceso, de modo que sirva como referencia para un futuro despliegue en un servidor de producción.

Estructura de archivos Docker

Se crearon tres archivos de configuración ubicados en las siguientes rutas del proyecto:

Archivo	Ubicación
`docker-compose.yml`	Raíz del proyecto
`Dockerfile` (backend)	`app-backend/Dockerfile`
`Dockerfile` (frontend)	`app-frontend/Dockerfile`
`nginx.conf`	`app-frontend/nginx.conf`

Dockerfile del backend

El Dockerfile del backend parte de la imagen oficial `python:3.13-slim`. Instala `uv` como gestor de dependencias, sincroniza los paquetes de producción y define un comando de inicio que encadena tres pasos: aplicar las migraciones con Alembic, ejecutar el script de datos iniciales (`seed.py`) y levantar el servidor Litestar.

```
FROM python:3.13-slim
WORKDIR /app

RUN pip install uv
COPY pyproject.toml uv.lock ./
RUN uv sync --frozen --no-dev

COPY . .
CMD ["sh", "-c", \
"uv run alembic upgrade head && \
uv run python seed.py && \
```

```
uv run litestar run --host 0.0.0.0 --port 8000"]
```

El flag `--frozen` asegura que las versiones instaladas coincidan exactamente con el lockfile, y `--no-dev` excluye las dependencias de desarrollo para reducir el tamaño de la imagen.

Dockerfile del frontend

El frontend utiliza una estrategia de *multi-stage build*. En la primera etapa se compila el proyecto Vue con Bun, y en la segunda se copian los archivos estáticos resultantes a una imagen ligera de Nginx que los sirve en el puerto 80.

```
FROM oven/bun:1 AS builder
WORKDIR /app
COPY package.json bun.lock ./
RUN bun install --frozen-lockfile
COPY . .
RUN bun run build

FROM nginx:alpine
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=builder /app/dist /usr/share/nginx/html
EXPOSE 80
```

Es necesario incluir una configuración personalizada de Nginx (`app-frontend/nginx.conf`) para que Vue Router funcione correctamente en modo `history`. Sin esta configuración, cualquier ruta distinta a la raíz devuelve un error 404 al recargar la página, ya que Nginx intenta buscar un archivo que no existe en disco.

```
server {
  listen 80;
  server_name localhost;
  root /usr/share/nginx/html;
  index index.html;

  location / {
    try_files $uri $uri/ /index.html;
  }
}
```

El comando `try_files` le indica a Nginx que, si no encuentra el archivo solicitado, redirija la petición a `index.html`, donde Vue Router se encarga de resolver la ruta del lado del cliente.

Orquestación con Docker Compose

El archivo `docker-compose.yml` define tres servicios que se levantan en orden gracias al comando `depends_on`:

1. **db**: contenedor de PostgreSQL 16 con un volumen persistente (`postgres_data`) que preserva los datos entre reinicios.
2. **backend**: construye la imagen desde `./app-backend`, expone el puerto 8000 y lee sus variables de entorno desde `app-backend/.env`.
3. **frontend**: construye la imagen desde `./app-frontend`, expone el puerto 80 y depende del backend.

```
services:
db:
image: postgres:16
restart: unless-stopped
env_file: ./app-backend/.env
volumes:
- postgres_data:/var/lib/postgresql/data
ports:
- "5432:5432"

backend:
build: ./app-backend
restart: unless-stopped
env_file: ./app-backend/.env
depends_on:
- db
ports:
- "8000:8000"

frontend:
build: ./app-frontend
restart: unless-stopped
depends_on:
```

```
    - backend
    ports:
    - "80:80"

    volumes:
    postgres_data:
```

Un aspecto importante del diseño es que tanto el servicio `db` como el `backend` comparten el mismo archivo `.env`. PostgreSQL lee automáticamente las variables `POSTGRES_USER`, `POSTGRES_PASSWORD` y `POSTGRES_DB` para inicializar la base de datos, mientras que el backend utiliza `DATABASE_URL` y las demás variables de configuración. Esto centraliza las credenciales en un solo lugar y evita la duplicación de datos sensibles.

Variables de entorno y seguridad

Para evitar que datos sensibles queden expuestos en el repositorio, se adoptó la convención de mantener un archivo `.env.example` versionado con los nombres de las variables pero sin valores reales, y un archivo `.env` ignorado por Git que contiene los valores efectivos. El `.env` del backend para Docker incluye las siguientes variables:

Variable	Descripción
`POSTGRES_USER`	Usuario de PostgreSQL.
`POSTGRES_PASSWORD`	Contraseña de PostgreSQL.
`POSTGRES_DB`	Nombre de la base de datos.
`DATABASE_URL`	Cadena de conexión completa utilizada por SQLAlchemy. Debe coincidir con las tres variables anteriores.
`SECRET_KEY`	Clave secreta para la firma de tokens JWT. Se recomienda generar un valor aleatorio con `python -c ïmport secrets; print(secrets.token_urlsafe(32))"`.
`DEBUG`	Activa el modo de depuración de Litestar (`true`/`false`).
`CORS_ALLOWED_ORIGINS`	Lista JSON de orígenes permitidos para CORS.

Conflictos encontrados y consideraciones para producción

Durante el proceso de contenedorización se identificaron varios problemas que es importante documentar para quienes deseen retomar el despliegue con Docker o llevarlo a un servidor real.

Conflicto de puertos con PostgreSQL. Si la máquina anfitriona ya tiene una instancia local de PostgreSQL corriendo en el puerto 5432, Docker no puede vincular ese mismo puerto y la aplicación falla al iniciar. La solución es cambiar el mapeo en `docker-compose.yml` a un puerto alternativo (por ejemplo, `"5433:5432"`) o detener el servicio local de PostgreSQL antes de levantar los contenedores.

Persistencia del volumen de datos. PostgreSQL solo ejecuta la inicialización (creación de usuario, base de datos) la primera vez que se crea el volumen. Si se cambian las credenciales en el `.env` después de la primera ejecución, los nuevos valores no tendrán efecto. En ese caso es necesario eliminar el volumen con `docker compose down -v` antes de volver a levantar, teniendo en cuenta que esto borra todos los datos almacenados.

CORS y comunicación entre servicios. Al servir el frontend desde Nginx en el puerto 80 y el backend en el puerto 8000, el navegador aplica la política de mismo origen (*Same-Origin Policy*). Aunque ambos servicios corren en `localhost`, los puertos diferentes hacen que el navegador los trate como orígenes distintos. Es imprescindible que la variable `CORS_ALLOWED_ORIGINS` incluya exactamente el origen desde el cual el usuario accede al frontend. En un servidor de producción, este valor debe actualizarse al dominio real (por ejemplo, `https://labdic.umag.cl`).

Adicionalmente, se detectó que la forma en que `pydantic-settings` parsea la variable `CORS_ALLOWED_ORIGINS` desde el archivo `.env` puede generar comportamientos inesperados: en lugar de interpretar el valor como una lista JSON, lo puede tratar como un único string, haciendo que Litestar no reconozca el origen permitido. Se recomienda verificar el parseo correcto mediante una prueba desde el contenedor antes de asumir que la configuración es válida.

Routing del frontend (SPA). Sin una configuración personalizada de Nginx, las rutas de Vue Router como `/login` o `/productos` devuelven un error 404 al recargar la página, ya que Nginx busca archivos literales en disco. La solución es incluir el archivo `nginx.conf` descrito anteriormente, que redirige todas las rutas a `index.html`.

Consideraciones adicionales para un servidor de producción. Para un despliegue real se deberían abordar, al menos, los siguientes aspectos que

quedaron fuera del alcance de este proyecto:

- Configurar HTTPS mediante un proxy inverso (por ejemplo, Nginx con certificados de Let's Encrypt o Caddy).
- Reemplazar las credenciales por defecto por valores seguros generados aleatoriamente.
- Restringir la exposición de puertos: el puerto de PostgreSQL no debería ser accesible desde fuera del entorno Docker.
- Implementar *health checks* en Docker Compose para que el backend espere a que PostgreSQL esté listo antes de ejecutar las migraciones.
- Evaluar el uso de un servidor ASGI de producción como Uvicorn con múltiples workers en lugar del servidor de desarrollo de Litestar.

10. Trabajo futuro

10.1. Mejoras al proyecto actual

Las siguientes mejoras corresponden a funcionalidades identificadas durante el desarrollo que quedaron fuera del alcance de la titulación. Se recomienda abordarlas como primera etapa de cualquier proyecto de continuación, ya que complementan directamente el módulo de préstamos ya implementado.

Seguridad y autenticación

Refresh Tokens

La implementación actual obliga al usuario a volver a iniciar sesión cada vez que el token JWT expira. Se propone implementar un mecanismo de refresh tokens que renueve automáticamente la sesión sin interrumpir el flujo de trabajo. Esto requiere configurar Litestar para emitir refresh tokens junto al access token, y modificar el interceptor de `apiFetch` en el frontend para intentar el refresco automático ante un error `401` antes de redirigir al login.

Expiración automática de cuentas

El campo `is_active` actualmente es controlado de forma manual por el administrador. Una mejora sería vincularlo a reglas automáticas, como desactivar cuentas que no han tenido actividad durante un período definido, o asignar fecha de vencimiento a cuentas temporales de alumnos de semestres específicos.

Módulo de Préstamos

Cancelación de solicitudes por el usuario

Actualmente el usuario no puede cancelar una solicitud una vez enviada. Se propone permitir la cancelación mientras la solicitud esté en estado `"pendiente"`, antes de que el administrador la revise.

Catálogo agrupado por producto

La vista actual del catálogo muestra una fila por cada dispositivo físico individual, lo que expone códigos internos que no son relevantes para el usuario. Se propone agrupar los dispositivos disponibles por producto, mostrando solo el nombre del producto con un contador de unidades disponibles. La asignación del dispositivo físico específico quedaría a criterio del administrador al registrar la entrega.

Serialización correcta del DTO de préstamos

La relación `device.product` dentro de los `loan_request_items` no se serializa correctamente desde el DTO del backend. Actualmente se resuelve con llamadas adicionales desde el frontend. Se propone resolver este problema a nivel del `LoanRequestReadDTO` para eliminar las llamadas redundantes.

Módulo de Dispositivos

`DeviceForm` *reutilizable*

El formulario de creación de dispositivos está embebido en `DevicesListView`. Se propone extraerlo a un componente independiente `DeviceForm.vue` para poder reutilizarlo desde `ProductDetailView` mediante un `Dialog` con el `productId` precargado.

Botones de `ProductDetailView`

El botón "Editar producto" actualmente navega a la lista de productos en lugar de abrir el Drawer de edición directamente. El botón "Agregar dispositivo" muestra un mensaje de próximamente en lugar de abrir un formulario. Ambos deben ser implementados correctamente.

Módulo de Usuarios

Drawer para ver detalles de usuario

El botón "Ver detalles" en `UsersListView` abre un Dialog simple. Se propone reemplazarlo por un Drawer lateral con el mismo diseño que el Drawer de detalle

de préstamos, mostrando toda la información del usuario de forma más cómoda y consistente.

Perfil personalizable en `MyUserView`

Se propone extender la vista de perfil personal con dos funcionalidades: una foto de perfil de tamaño reducido (almacenada como `base64` o referencia de archivo con límite de tamaño), y un sistema de secciones académicas personalizables donde el usuario puede completar campos como descripción personal, proyectos desarrollados, asignaturas cursando, aprobadas, favoritas y no favoritas con justificación. Estas secciones podrían almacenarse como un campo JSON en el modelo `User` del backend para mayor flexibilidad.

10.2. Nuevos módulos propuestos

Los siguientes módulos representan extensiones naturales de la plataforma hacia otras necesidades del LabDIC. Cada uno puede desarrollarse de forma independiente como proyecto de titulación, conectándose a la infraestructura existente.

10.2.1. Préstamo de Inventario (continuación)

Corresponde a la consolidación de todas las mejoras pendientes descritas en la sección 10.1. Incluye la resolución de los problemas técnicos identificados, la implementación de las funcionalidades que quedaron fuera del alcance y la incorporación de mejoras de UX identificadas durante el desarrollo.

Posible alcance

- Todas las mejoras de la sección anterior de este documento.
- Implementación de refresh tokens y expiración automática de cuentas.
- Catálogo de dispositivos agrupado por producto.
- Perfil de usuario con foto y secciones académicas.
- Cancelación de solicitudes de préstamo por el usuario.
- Optimización de DTOs del backend para eliminar llamadas redundantes.

10.2.2. Administración de Inventario

Módulo orientado a tareas administrativas del laboratorio que van más allá de los préstamos. Toma como base el modelo de datos ya desarrollado (productos, dispositivos, ubicaciones) y agrega herramientas de gestión documental y reportería para los administradores del LabDIC.

Posible alcance

- Dashboard administrativo con estadísticas del inventario: dispositivos por estado, por ubicación y por categoría.
- Exportación de reportes en formatos `.xlsx`, `.pdf` y `.docx` con el inventario completo o filtrado.
- Generación de documentos internos administrativos: memorándum de baja de dispositivos y memorándum de traslado de dispositivos entre ubicaciones.
- Historial completo de movimientos de cada dispositivo con trazabilidad por usuario.
- Alertas de dispositivos en mantención prolongada o con préstamos vencidos.
- Gestión de proveedores y registro de adquisiciones de equipos.

10.2.3. Gestión de DHCP

Los administradores del LabDIC gestionan manualmente el archivo de configuración DHCP para controlar qué dispositivos externos pueden conectarse a la red WiFi del laboratorio. Este módulo propone una interfaz que abstraiga la manipulación directa del archivo DHCP, mejorando significativamente la experiencia del administrador y reduciendo errores de configuración.

Contexto técnico

El sistema DHCP manual del laboratorio asigna direcciones IP fijas a dispositivos identificados por su dirección MAC. Las direcciones están organizadas en rangos IPv4 según el tipo de dispositivo y la sala a la que pertenecen: dispositivos inalámbricos temporales de estudiantes, equipos fijos de salas TI-1, TI-2, Redes, entre otros.

Posible alcance

- Interfaz web para agregar, editar y eliminar registros DHCP sin editar el archivo de configuración directamente.
- Formulario de registro con: nombre de usuario, nombre del dispositivo, dirección MAC, tipo de dispositivo y sala o rango IPv4 de destino.
- Validación automática de MAC duplicadas antes de agregar un nuevo registro.
- Control de cuota por tipo de usuario: máximo de dispositivos configurables por rol (por ejemplo, 3 para usuario común, ilimitados para profesores y administradores).
- Asignación de fecha de expiración para dispositivos temporales de estudiantes por semestre, con eliminación automática al vencer.
- Visualización del estado actual del archivo DHCP organizado por rangos IPv4 y salas.
- Exportación del archivo de configuración DHCP listo para aplicar en el servidor.

Este módulo tiene complejidad técnica significativa por su interacción con la infraestructura de red del laboratorio. Se recomienda como proyecto de titulación independiente con participación del equipo de administración de redes del departamento.

10.2.4. Administración de Salas

Módulo de gestión de horarios y reservas de las salas de clases del LabDIC. Permite a los profesores solicitar salas, visualizar disponibilidad en formato calendario y gestionar cambios de sala, todo bajo supervisión y aprobación de los administradores del laboratorio.

Contexto

El LabDIC administra varias salas de clases: TI-1, TI-2, Redes, Magister, entre otras. La coordinación de horarios y asignación de salas actualmente se realiza de forma manual, lo que puede generar conflictos y falta de visibilidad para los profesores.

Posible alcance

- Vista de calendario por sala con columnas por día de la semana (lunes a viernes) y filas por bloque horario de 1 hora 30 minutos desde las 8:00 hasta las 19:20.
- Un calendario independiente por cada sala: TI-1, TI-2, Redes, Magister y otras que se agreguen.
- Nuevo rol "Profesor" con permisos específicos: visualizar calendarios, solicitar asignación de sala y solicitar cambios.
- Flujo de asignación de sala: el profesor solicita una sala para un horario específico, el administrador aprueba o rechaza con motivo.
- Solicitud de cambio de sala entre profesores (*swap*): ambos profesores deben aceptar el intercambio antes de que el administrador lo apruebe.
- Solicitud de sala temporal por período menor a un mes para actividades extraordinarias.
- Vista de administrador con control total: puede asignar, modificar y eliminar cualquier reserva.
- Notificaciones o alertas cuando se aprueba o rechaza una solicitud.

Este módulo tiene alta complejidad de UX por la naturaleza del calendario interactivo y los múltiples flujos de aprobación. Se recomienda como proyecto de titulación independiente con participación directa de los profesores del departamento como usuarios de prueba.

10.3. Resumen y recomendaciones

La tabla 16 resume los cuatro módulos propuestos con su clasificación de complejidad y el tipo de proyecto que mejor se adapta a cada uno.

Módulo	Tipo	Recomendación
Préstamo de Inventario (continuación)	Continuación directa	Primera etapa para cualquier proyecto de continuación.
Administración de Inventario	Extensión nueva	Puede combinarse con el módulo 1 en una sola titulación.
Gestión de DHCP	Módulo nuevo	Titulación independiente con foco en infraestructura de red.
Administración de Salas	Módulo nuevo	Titulación independiente con foco en UX y flujos de aprobación.

Cuadro 16: Resumen de módulos de trabajo futuro propuestos.

Se recomienda que el primer proyecto de continuación aborde en conjunto los módulos 1 y 2, consolidando el trabajo existente y añadiendo las capacidades administrativas de reportería. Los módulos 3 y 4, por su complejidad técnica y su alcance, se recomiendan como proyectos de titulación independientes, idealmente con participación activa del personal del LabDIC desde las etapas tempranas de análisis de requerimientos.

La arquitectura modular de la plataforma (Vue 3 + Litestar + PostgreSQL) fue diseñada deliberadamente para facilitar la incorporación de nuevos módulos sin reestructurar lo existente. Cada módulo nuevo se integra como un conjunto de rutas, controladores, servicios y vistas independientes que conviven con los módulos actuales.

11. Conclusión

La motivación de este proyecto nació de la observación directa de las ineficiencias en los procesos de gestión del LabDIC, donde la experiencia sostenida en el laboratorio permitió identificar con claridad los puntos críticos que una solución tecnológica podía resolver. La combinación entre formación académica e investigación aplicada fundamentó la selección de un stack tecnológico que satisface los requerimientos actuales y sienta las bases para un eventual despliegue en producción.

El modelo de base de datos evolucionó desde una concepción inicial de tres ejes funcionales hacia una arquitectura de seis módulos cohesivos, donde la mo-

dularidad adoptada es de naturaleza conceptual: no todos los módulos requieren tablas propias, sino que algunos se expresan a través de campos y relaciones dentro de tablas existentes. La integración de SQLAlchemy y Alembic proporcionó un marco robusto para la evolución iterativa del esquema, asegurando que cada cambio quedara documentado y fuera reproducible.

La documentación arquitectónica del sistema evidenció que las decisiones de diseño responden a principios consistentes de separación de responsabilidades aplicados de forma simétrica en ambas capas. En el backend, la división entre DTOs, repositorios y controladores garantiza que cada componente tenga una responsabilidad acotada; en el frontend, la cadena `types/` → `services/` → stores de Pinia replica esta misma lógica. Esta uniformidad hace que añadir un nuevo módulo implique replicar un patrón ya establecido.

La implementación del backend consolidó la API REST como una capa completa y estructurada. Los módulos más complejos, Dispositivos y Préstamos, ilustraron cómo la lógica de negocio puede encapsularse en el repositorio dejando los controladores livianos. Las migraciones generadas durante este proceso confirman que el desarrollo del backend no es un evento único, sino un proceso iterativo que evoluciona en paralelo con las demás capas del sistema.

El frontend completó el ciclo de desarrollo mediante seis fases progresivas que construyeron la interfaz de forma incremental. La interdependencia real entre frontend y backend quedó en evidencia cuando la implementación de vistas reveló la necesidad de correcciones en cascada en la base de datos. Desde la perspectiva del usuario, el sistema resultante ofrece una experiencia diferenciada según el rol, con soporte para modo claro y oscuro, diseño *responsive* y retroalimentación visual mediante notificaciones. Con todas las capas implementadas, la documentación del trabajo futuro identifica mejoras pendientes y propone nuevos módulos que la arquitectura modular de la plataforma está preparada para incorporar.

Bibliografía

[1] Amazon Web Services. «MySQL vs PostgreSQL: What's the Difference?» Accedido: 27 de agosto de 2025. (2023), dirección: `https://aws.amazon.com/es/compare/the-difference-between-mysql-vs-postgresql/`.

[2] pawnsapprentice. «Understanding PostgreSQL and Its Strong Embrace of ACID Properties.» Accedido: 27 de agosto de 2025. (2021), dirección:

https://dev.to/pawnsapprentice/understanding-postgresql-and-its-strong-embrace-of-acid-properties-2dpl.

[3] Airbyte Team. «SQLite vs PostgreSQL: Which Database Should You Use?» Accedido: 27 de agosto de 2025. (2023), dirección: https://airbyte.com/data-engineering-resources/sqlite-vs-postgresql.

[4] Prisma Data Guide. «The Benefits of PostgreSQL.» Accedido: 27 de agosto de 2025. (2023), dirección: https://www.prisma.io/dataguide/postgresql/benefits-of-postgresql.

[5] 4Geeks Academy. «Todo lo necesario para empezar a usar SQLAlchemy.» Accedido: 4 de septiembre de 2025. (2023), dirección: https://4geeks.com/es/lesson/todo-lo-necesario-para-empezar-usar-sqlalchemy.

[6] HostGator Blog. «ORM: ¿Qué es y cuándo usarlo?» Accedido: 4 de septiembre de 2025. (2022), dirección: https://www.hostgator.cl/blog/orm-object-relational-mapper-que-es-y-cuando-usarlo/.

[7] Auth0. «SQLAlchemy ORM Tutorial for Python Developers.» Accedido: 4 de septiembre de 2025. (2021), dirección: https://auth0.com/blog/sqlalchemy-orm-tutorial-for-python-developers/.

[8] LearnPython.com. «Learn Python: Why Python's Popularity Has Soared.» Accedido: 10 de septiembre, 2025. (2023), dirección: https://learnpython.com/blog/python-popularity/.

[9] Anaconda. «Why Python? 10 Reasons Behind Its Popularity.» Accedido: 10 de septiembre, 2025. (2024), dirección: https://www.anaconda.com/blog/why-python.

[10] SQLAlchemy Project. «SQLAlchemy: The Database Toolkit for Python — Features.» Accedido: 4 de septiembre de 2025. (2025), dirección: https://www.sqlalchemy.org/features.html.

[11] PingCAP. «Best Practices for Alembic Schema Migration.» Accedido: 27 de agosto de 2025. (2023), dirección: https://www.pingcap.com/article/best-practices-alembic-schema-migration/.

[12] VivekTheDev. «Effortless Database Migrations: Why Alembic is Your Python Must-Have.» Accedido: 27 de agosto de 2025. (2022), dirección: https://dev.to/vivekthedev/effortless-database-migrations-why-alembic-is-your-python-must-have-2f0n.

[13] BetterStack. «Litestar to FastAPI Migration Guide.» Accedido: 10 de septiembre, 2025. (2024), dirección: https://betterstack.com/community/guides/scaling-python/litestar-to-fastapi-migration/.

[14] P. English. «Litestar vs FastAPI: The Next Big Thing in Python API Frameworks.» Accedido: 10 de septiembre, 2025. (2023), dirección: https://python.plainenglish.io/litestar-vs-fastapi-the-next-big-thing-in-python-api-frameworks-107a8f44ebe7.

[15] L. Documentation. «Advanced Alchemy with Litestar.» Accedido: 10 de septiembre, 2025. (2025), dirección: https://docs.advanced-alchemy.litestar.dev/latest/usage/frameworks/litestar.html.

[16] L. Documentation. «Dependency Injection in Litestar.» Accedido: 10 de septiembre, 2025. (2025), dirección: https://docs.litestar.dev/2/usage/dependency-injection.html.

[17] L. Documentation. «Abstract Authentication Middleware in Litestar.» Accedido: 10 de septiembre, 2025. (2025), dirección: https://docs.litestar.dev/main/usage/security/abstract-authentication-middleware.html.

[18] I. T. Institute. «Vue 3: todas las novedades del framework frontend.» Accedido: 10 de septiembre, 2025. (2023), dirección: https://immune.institute/blog/novedades-vue-3-framework/.

[19] Acrocommerce. «TypeScript Benefits and Best Practices in Web Development.» Accedido: 10 de septiembre, 2025. (2023), dirección: https://www.acrocommerce.com/article/typescript-benefits-and-best-practices-in-web-development.

[20] Strapi. «Benefits of TypeScript.» Accedido: 10 de septiembre, 2025. (2022), dirección: https://strapi.io/blog/benefits-of-typescript.

[21] PrimeVue. «Introduction to PrimeVue.» Accedido: 10 de septiembre, 2025. (2025), dirección: https://primevue.org/introduction/.

[22] T. M. Reality. «Top 5 Vue.js Libraries in 2025.» Accedido: 10 de septiembre, 2025. (2025), dirección: https://themobilereality.com/blog/javascript/top-5-vue-js-libraries-in-2025.

[23] PrimeVue. «Tailwind CSS Presets for PrimeVue.» Accedido: 10 de septiembre, 2025. (2025), dirección: https://primevue.org/tailwind/.

[24] TailwindCSS. «Optimizing for Production.» Accedido: 10 de septiembre, 2025. (2025), dirección: https://v3.tailwindcss.com/docs/optimizing-for-production.

[25] ScriptBinary. «Scalable Tailwind CSS Best Practice.» Accedido: 10 de septiembre, 2025. (2025), dirección: https://scriptbinary.com/tailwind/scalable-tailwind-css-best-practices.

[26] Syncfusion. «Vite.js: A Faster Frontend Development Experience.» Accedido: 10 de septiembre, 2025. (2022), dirección: https://www.syncfusion.com/blogs/post/vitejs-faster-frontend-development.

[27] A. Sharma. «Vite vs Webpack: Which One is Right for Your Project?» Accedido: 10 de septiembre, 2025. (2023), dirección: https://dev.to/abhinav_sharma_e01f930be6/vite-vs-webpack-which-one-is-right-for-your-project-886.

[28] R. Ivi. «State Management with Pinia vs Vuex.» Accedido: 10 de septiembre, 2025. (2023), dirección: https://dev.to/robin-ivi/state-management-with-pinia-vs-vuex-4mh.

[29] M. Solutions. «Optimizing Vue.js Apps in 2025: Lazy Loading, Tree Shaking, and More.» Accedido: 10 de septiembre, 2025. (2025), dirección: https://metadesignsolutions.com/optimizing-vuejs-apps-in-2025-lazy-loading-tree-shaking-more/.

[30] Astral. «Introducing uv: the Python package manager reimagined.» Accedido: 10 de septiembre, 2025. (2023), dirección: https://astral.sh/blog/uv.

[31] DigitalOcean. «UV: Python Package Manager.» Accedido: 10 de septiembre, 2025. (2024), dirección: https://www.digitalocean.com/community/conceptual-articles/uv-python-package-manager.

[32] Xebia. «UV: The Engineering Secrets Behind Python's Speed King.» Accedido: 10 de septiembre, 2025. (2024), dirección: https://xebia.com/blog/uv-the-engineering-secrets-behind-pythons-speed-king/.

[33] S. Pegasus. «UV Deep Dive Guide.» Accedido: 10 de septiembre, 2025. (2024), dirección: https://www.saaspegasus.com/guides/uv-deep-dive/.

[34] PyDevTools. «UV in Production.» Accedido: 10 de septiembre, 2025. (2024), dirección: https://pydevtools.com/blog/uv-in-prod/.

[35] Strapi. «Bun vs Node.js: Performance Comparison Guide.» Accedido: 10 de septiembre, 2025. (2023), dirección: https://strapi.io/blog/bun-vs-nodejs-performance-comparison-guide.

[36] DreamHost. «Bun vs Node.js: Guía completa.» Accedido: 10 de septiembre, 2025. (2024), dirección: https://www.dreamhost.com/blog/es/bun-vs-node-guia-completa/.

[37] KeepCoding. «Bun JavaScript Runtime: sus ventajas.» Accedido: 10 de septiembre, 2025. (2023), dirección: https://keepcoding.io/blog/bun-javascript-runtime-sus-ventaja/.

[38] MilesWeb. «NPM vs Bun.» Accedido: 10 de septiembre, 2025. (2024), dirección: https://www.milesweb.co.uk/blog/technology-hub/npm-vs-bun/.

[39] GitHub. «Git Handbook.» Accedido: 10 de septiembre, 2025. (2025), dirección: https://guides.github.com/introduction/git-handbook/.

[40] A. Systems. «Despliegue de aplicaciones.» Accedido: 10 de septiembre, 2025. (2023), dirección: https://aidomoticssystems.com/index.php/2023/06/23/despliegue-de-aplicaciones/.

[41] KeepCoding. «Despliegue de aplicaciones locales con Docker.» Accedido: 10 de septiembre, 2025. (2023), dirección: https://keepcoding.io/blog/despliegue-de-aplicaciones-locales-con-docker/.

[42] Hostinger. «¿Qué es Docker?» Accedido: 10 de septiembre, 2025. (2024), dirección: https://www.hostinger.com/es/tutoriales/que-es-docker.

[43] Editverse. «De cero a héroe: configuración de microservicios en contenedores con Docker.» Accedido: 10 de septiembre, 2025. (2024), dirección: https://www.editverse.com/es/De-cero-a-h%C3%A9roe:-configuraci%C3%B3n-de-microservicios-en-contenedores-con-Docker/.

[44] M. Learn. «Orchestration in Microservices Design.» Accedido: 10 de septiembre, 2025. (2024), dirección: https://learn.microsoft.com/es-es/azure/architecture/microservices/design/orchestration.

[45] TildaVPS. «10 casos de uso poderosos de Docker para servidores dedicados.» Accedido: 10 de septiembre, 2025. (2024), dirección: https://tildavps.com/blog/es/10-casos-de-uso-poderosos-de-docker-para-servidores-dedicados-revolucione-su-infraestructura.

[46] T. Yamamoto. «Modernizing Database Interactions with SQLAlchemy 2.0 and Python Dataclasses.» Accedido: 7 de abril, 2026. (2025), dirección: https://leapcell.io/blog/modernizing-database-interactions-with-sqlalchemy-2-0-and-python-dataclasses.

[47] M.-j. Kim. «Decoupling API Layers with Pydantic Models for Robust Data Transfer.» Accedido: 7 de abril, 2026. (2025), dirección: https://leapcell.io/blog/decoupling-api-layers-with-pydantic-models-for-robust-data-transfer.

A. Evidencias de la implementación

A continuación se presentan las capturas de pantalla que evidencian el funcionamiento del sistema completo, organizadas según los módulos y vistas implementados en el frontend.

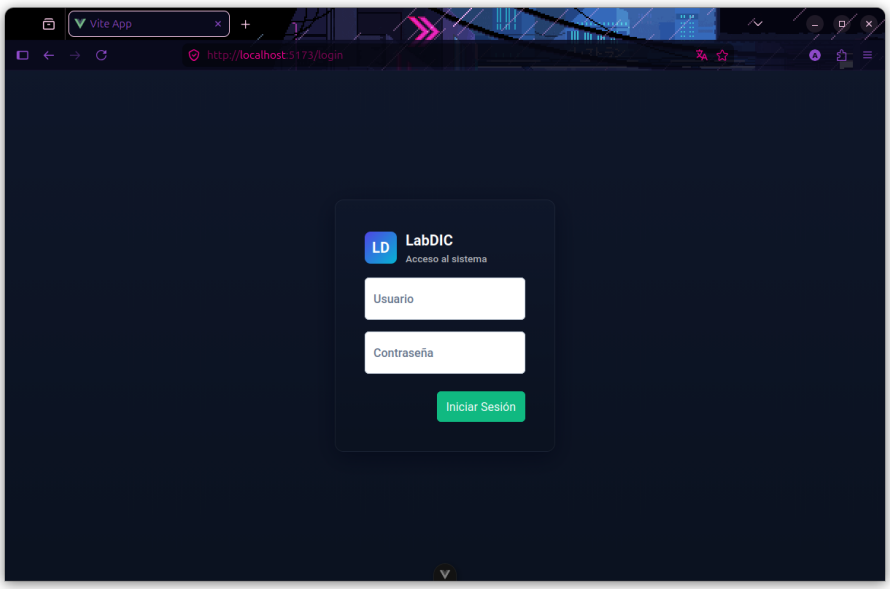

Figura 4: Vista de autenticación — pantalla de inicio de sesión

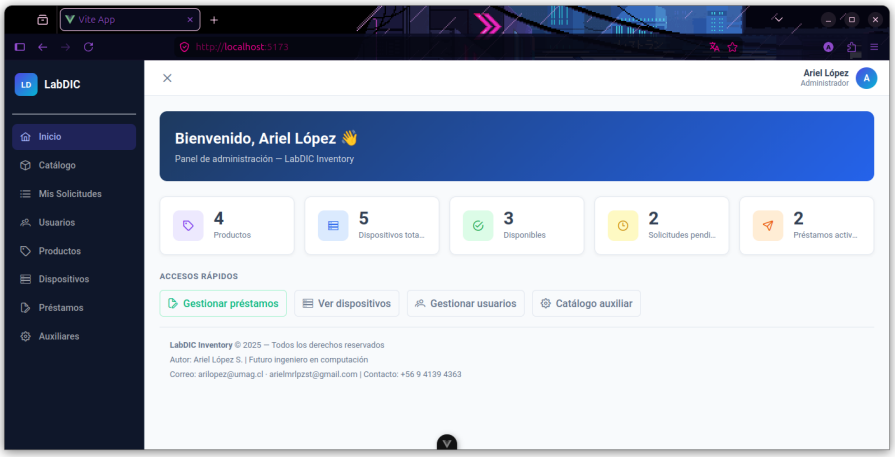

Figura 5: Vista de inicio para el rol administrador

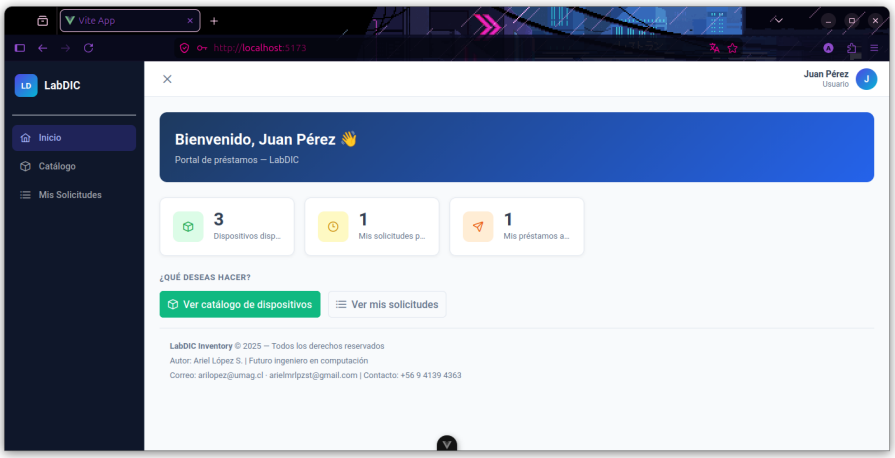

Figura 6: Vista de inicio para el rol usuario

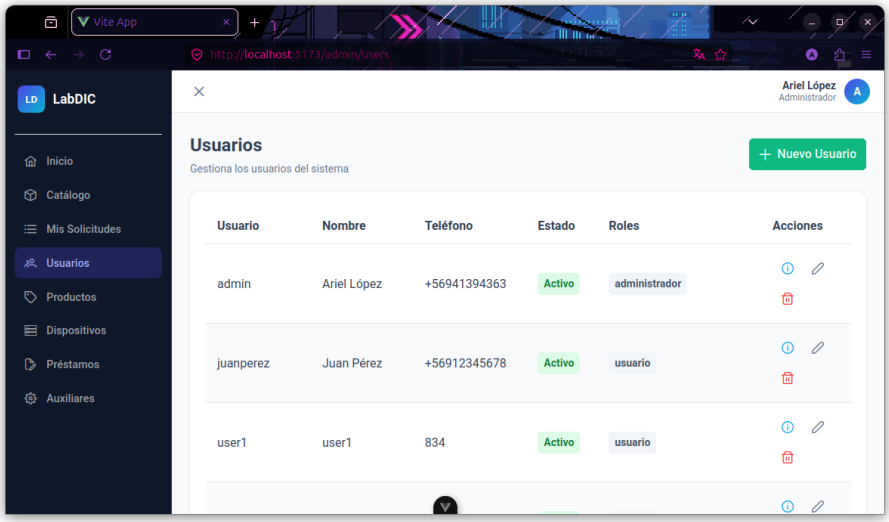

Figura 7: Vista de administración de usuarios

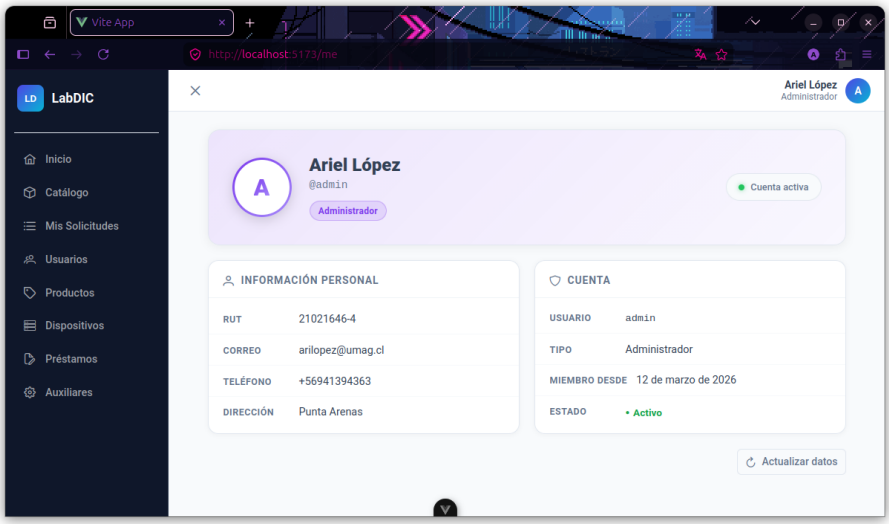

Figura 8: Vista de perfil del usuario autenticado

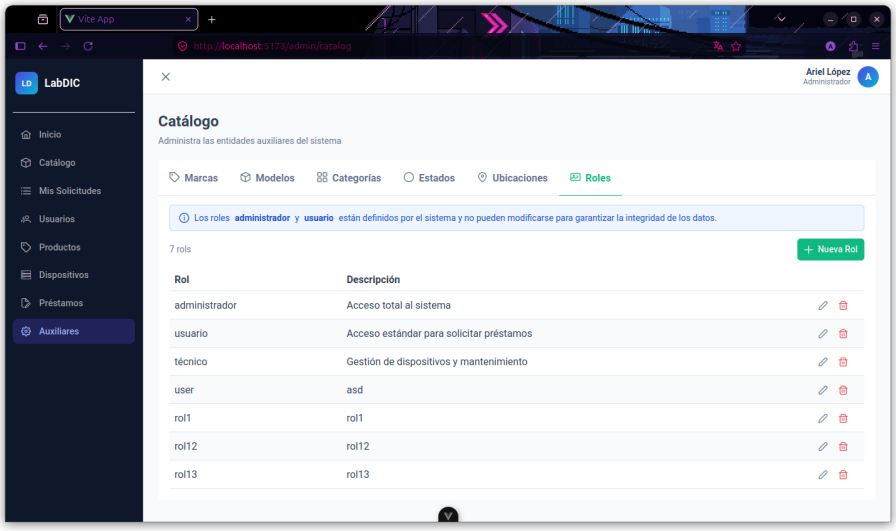

Figura 9: Vista de catálogo — sección de roles

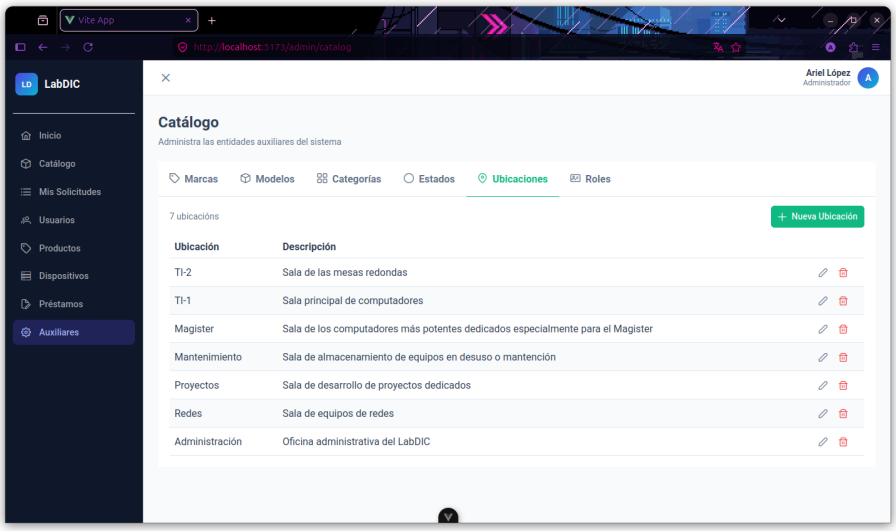

Figura 10: Vista de catálogo — sección de ubicaciones

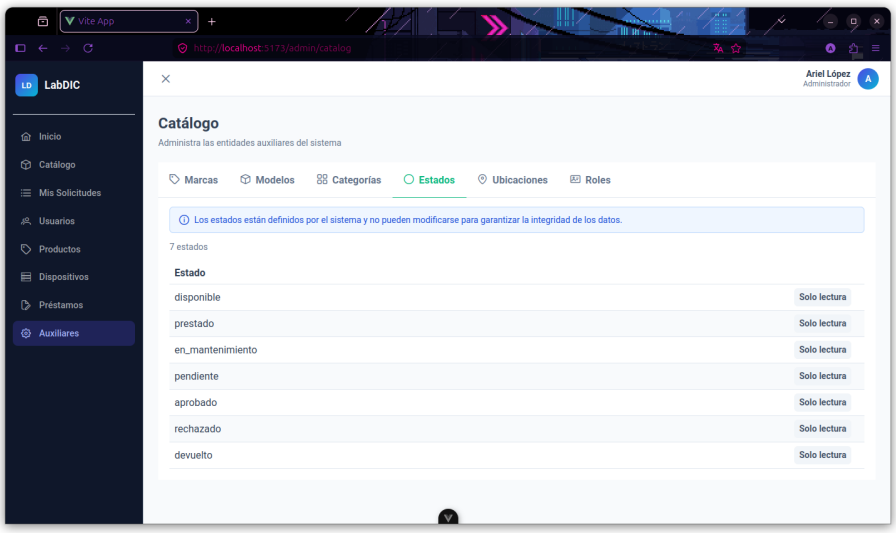

Figura 11: Vista de catálogo — sección de estados

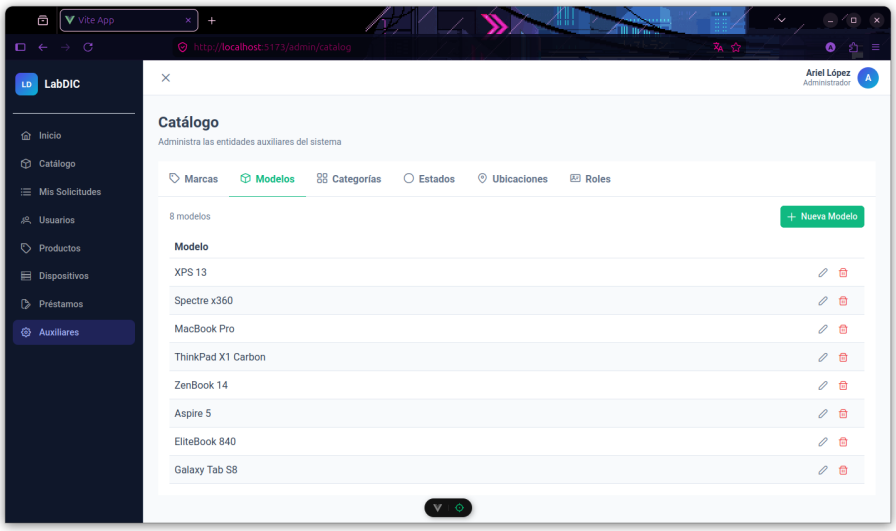

Figura 12: Vista de catálogo — sección de modelos

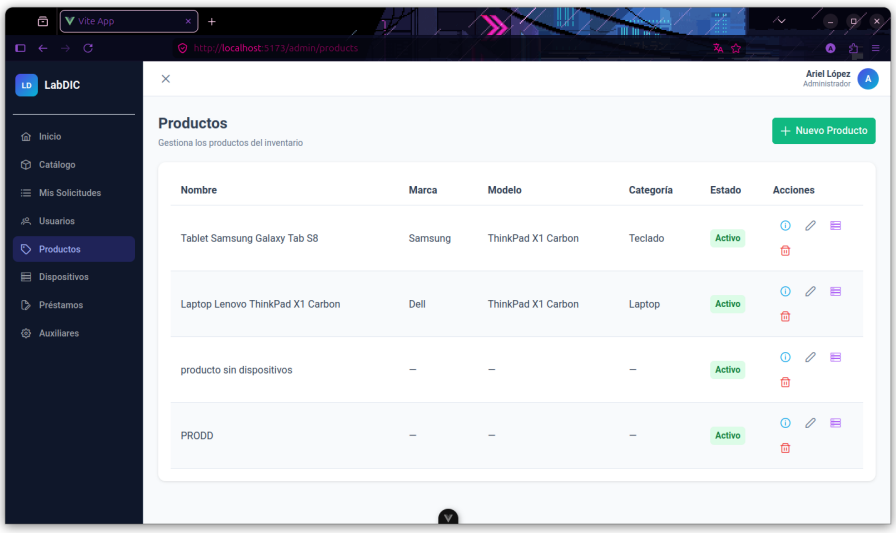

Figura 13: Vista de administración de productos

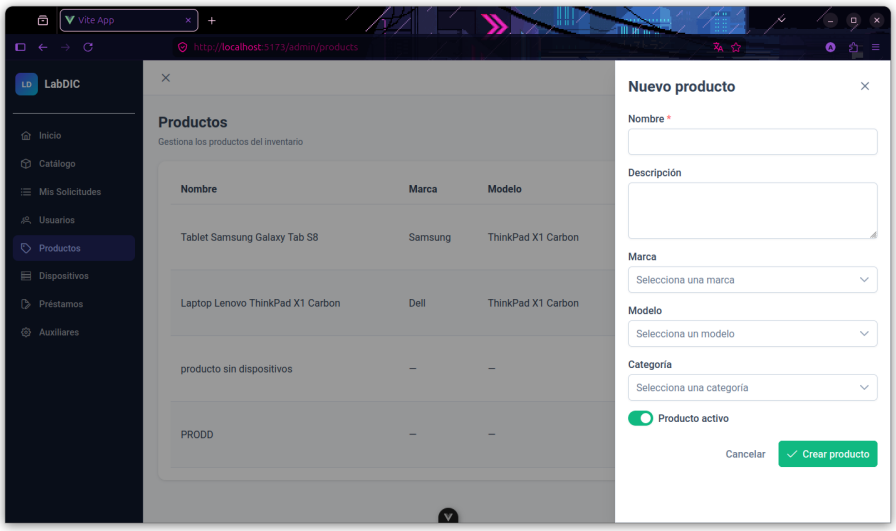

Figura 14: Vista de administración de productos — formulario de nuevo producto

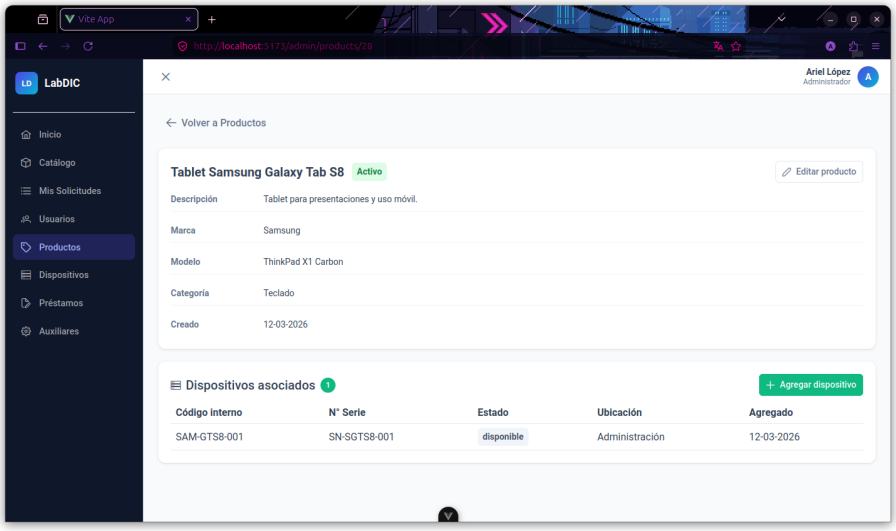

Figura 15: Vista de administración de productos — detalle de producto

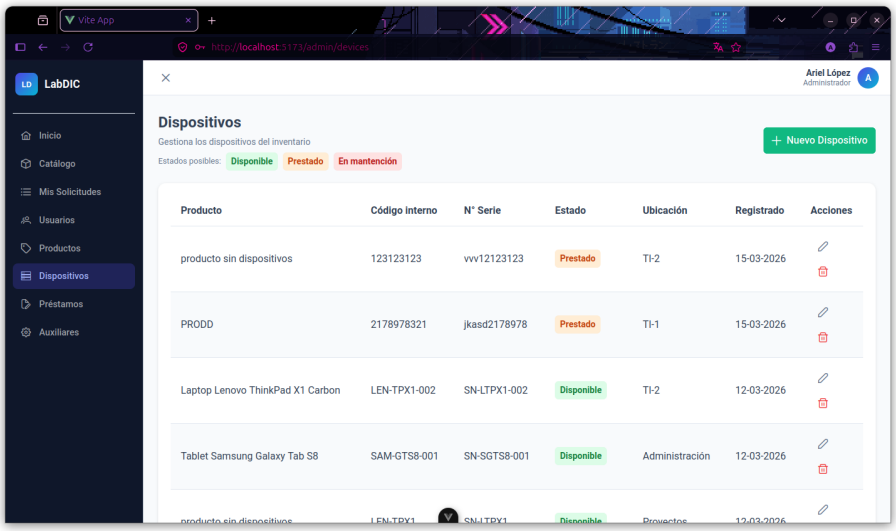

Figura 16: Vista de administración de dispositivos

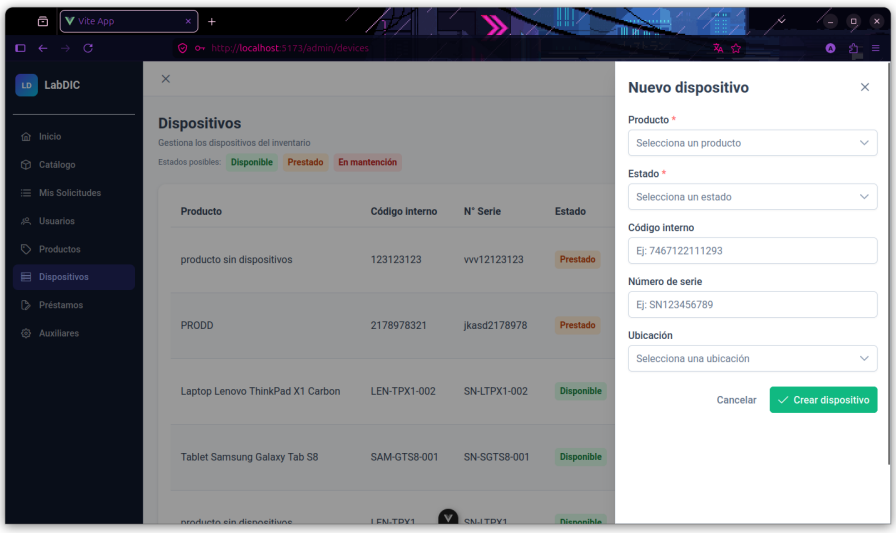

Figura 17: Vista de administración de dispositivos — formulario de nuevo dispositivo

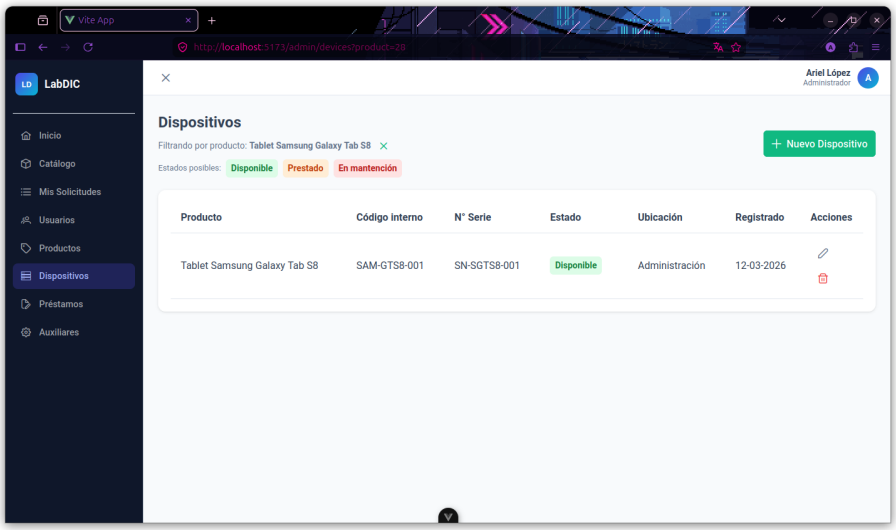

Figura 18: Vista de dispositivos filtrada desde detalle de producto

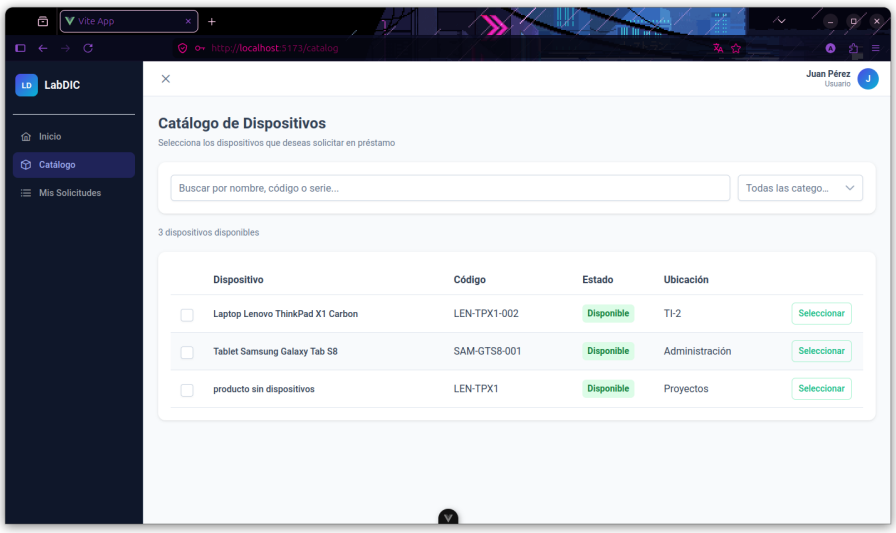

Figura 19: Vista de catálogo de dispositivos disponibles para usuarios

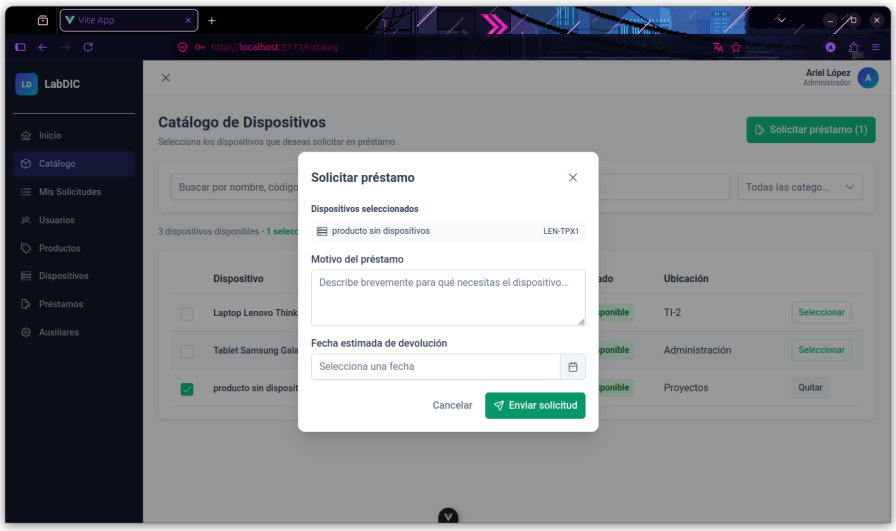

Figura 20: Vista de catálogo de dispositivos — formulario de solicitud de préstamo

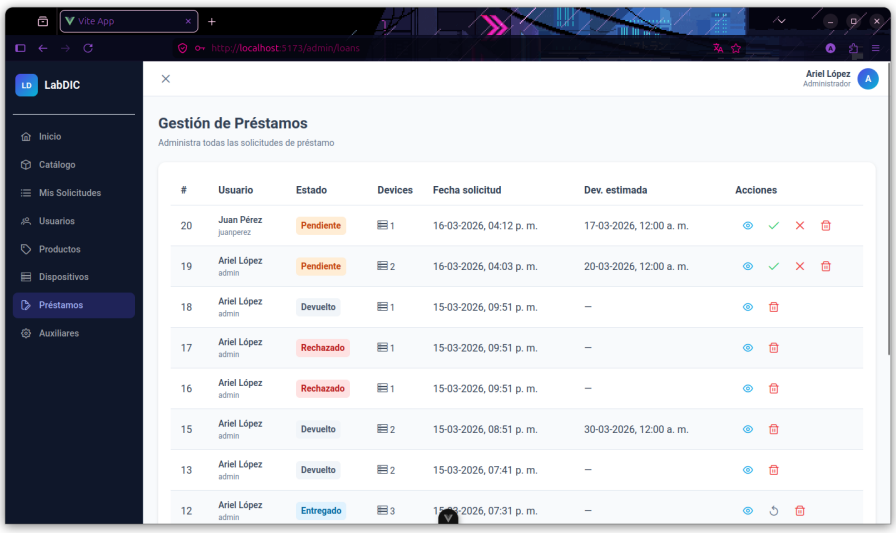

Figura 21: Vista de administración de solicitudes de préstamo

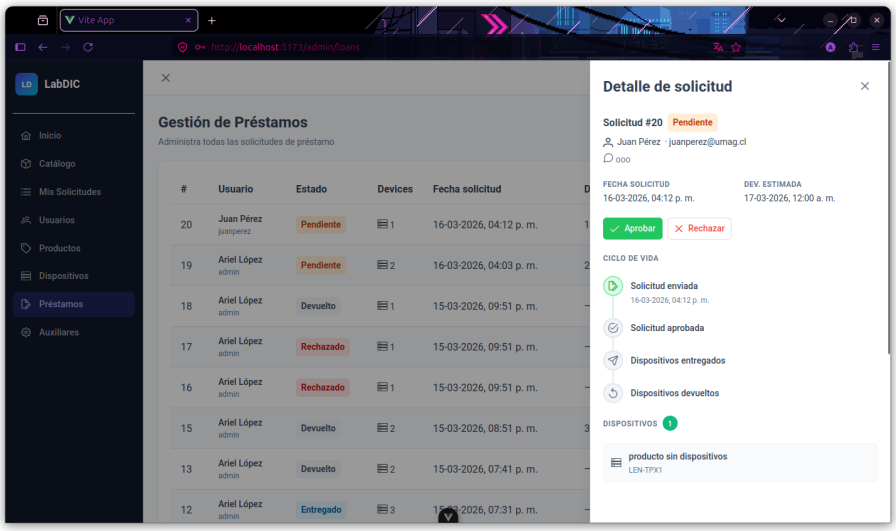

Figura 22: Vista de administración de préstamos — ciclo de vida en estado inicial

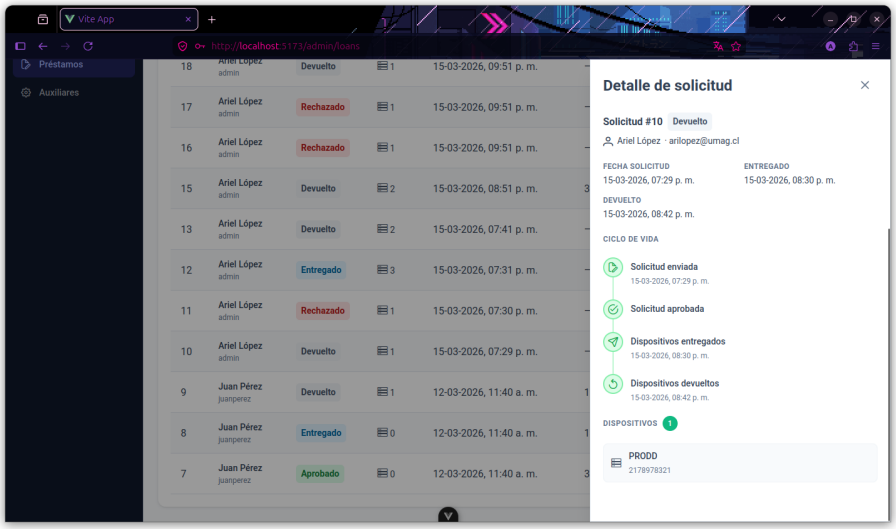

Figura 23: Vista de administración de préstamos — ciclo de vida finalizado

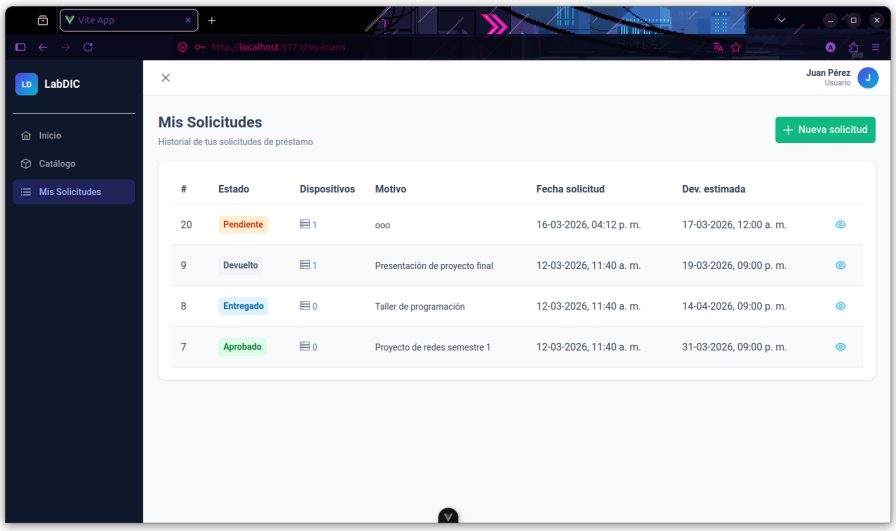

Figura 24: Vista de solicitudes del usuario autenticado

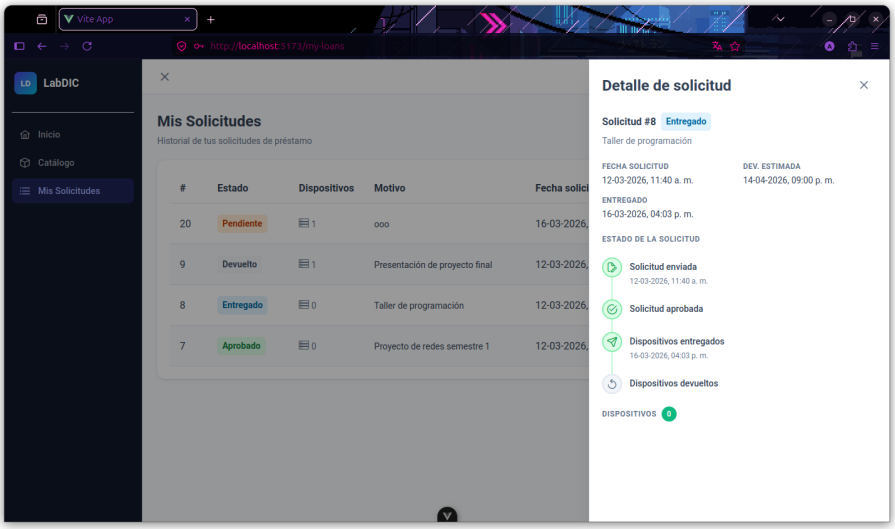

Figura 25: Vista de solicitudes del usuario autenticado — detalle de solicitud